



POLITECNICO
MILANO 1863

**SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE**

Lab report

Project Report - Course of Automation and Control Laboratory
Automation and Control Engineering

Francesco Berruti, 10777631;
Alessandro Donadi, 10711295;
Fausto Allegrini, 10979847;
Ahmad Labib, 10971357;
Lorenzo Marzi, 11015873;

Contents

1	Description of the System	4
2	Dynamical Model	4
2.1	Reference system	5
2.2	Lagrange equations	5
2.2.1	Computing the Lagrangian	5
2.2.2	Computing the Generalized Forces	6
2.3	Equations of motion	7
3	Propeller Dynamics and Control	7
3.1	Propeller Model	7
3.2	Propeller Control	8
3.3	Identification	9
3.4	Implementation results	9
4	Modelling the System in 2 DOF	10
4.1	State Space Representation of Mechanical System	10
4.2	State Space Representation of the Full System	10
4.3	Linearisation of the Full System	11
4.3.1	Linearisation of Mechanical System	12
5	1 DOF VTOL Modeling and Control	12
5.1	1 DOF Model	12
5.1.1	Linearisation	13
5.2	Parameter Identification	13
5.2.1	Force Coefficient	13
5.2.2	Damping	14
5.3	PID Control	14
5.4	Fuzzy PID Control	18
5.5	Model Predictive Control (MPC)	22
6	2 DOF Helicopter Control	23
6.1	Parameter Identification	24
6.1.1	Yaw Model Parameters	24
6.1.2	Coupling Torque Coefficients	25
6.2	PID Control with Decouplers	26
6.3	Control with LMIs	29
6.4	LQ Control	33
6.4.1	Controller dimensioning	33
6.4.2	Results	35
6.5	LQ-TV Control	36
6.5.1	Optimal Trajectory	36
6.5.2	Controller dimensioning	37
6.5.3	Results	37
6.6	LQG control	39
6.6.1	Kalman Filter	39
6.6.2	Kalman Filter dimensioning	39
6.7	Quasi-Infinite Horizon Nonlinear MPC with Stability Guarantees	42
6.7.1	Simulation and Real-Time Implementation	42
7	Controller performance comparisons	43
7.1	1 DOF performance comparison	43
7.2	2 DOF performance comparison	44

Notation

Symbol	Description	Numerical Value
J_p	Moment of inertia of the pitch	0.0232 kgm^2
J_y	Moment of inertia of the yaw	0.0238 kgm^2
J_{motor}	Moment of inertia of the motor	
β	Force model coefficient	$3.73 \text{ Nms}^2 \times 10^{-6}$
g	Acceleration due to gravity	9.81 m/s^2
β_θ	Pitch damping	$1.763 \times 10^{-3} \text{ Nms}$
β_ψ^+	Yaw damping with increasing yaw	0.0049 Nms
β_ψ^-	Yaw damping with decreasing yaw	0.0065 Nms
C_{fan}	Damping coefficient of the fan	
μ	Friction of the yaw	
μ_s	Static Friction of the yaw	0.0113 Nm
μ_d^+	Dynamic Friction with increasing yaw	0.007 Nm
μ_d^-	Dynamic Friction with decreasing yaw	0.0111 Nm
M_{body}	Mass of the arm	1.07 kg
D_m	Displacement center of mass	0.24 cm
L_m	Distance from motor to center	16.7 cm
R_a	Resistance of motor	$8.4 \text{ } \Omega$
L_a	Inductance of motor	1.16 mH
K_e	Back-EMF constant	0.042 Vs/rad
K_t	motor torque constant	0.042 Nms/rad
γ	Drag coefficient of the fan	

Table 1: List of symbols used in the report with their numerical values.

1. Description of the System

The Quanser AERO 2 system, see Figure 1, consists of two identical rotors attached symmetrically to a central body, which itself is fixed to a vertical support yoke. This vertical support is mounted on a stable base that contains all the necessary electronic components. The system represents a simplified helicopter model and thus allows motion in two principal axes: pitch (rotation of the body) and yaw (rotation about the support yoke).



Figure 1: quanser 2

A distinctive feature of the Quanser AERO 2 is its adaptability, allowing for multiple operational configurations, such as:

- 1 DOF VTOL (Vertical Take-Off and Landing): The pitch axis is free to rotate while the yaw axis is locked. Both rotors are oriented horizontally.
- 2 DOF Helicopter: Both pitch and yaw axes are unlocked, with the rear rotor oriented vertically.

The device is actuated by two direct-drive, 18 V brushed DC motors that power two eight-blade, counter-rotating, 3D-printed propellers.

The Quanser AERO 2 also incorporates three distinct encoders for precise position and velocity measurements:

- Optical shaft encoders measuring angular positions of the DC motors, each providing 2048 counts per revolution in quadrature mode.
- A single-ended optical shaft encoder for the pitch axis, offering 2880 counts per revolution in quadrature mode.
- An optical encoder dedicated to measuring the yaw position of the support yolk, providing 4096 counts per revolution in quadrature mode.

Further technical details and specifications are available in the official User Manual [4].

2. Dynamical Model

Here we present the mathematical model used to describe the motion of the Quanser AERO 2 system. The focus is on obtaining the two-degree-of-freedom (2 DOF) model, as the one-degree-of-freedom (1 DOF) model follows similar derivation steps, or alternatively, can be obtained from the 2 DOF model with some modifications.

The system consists of two rigid bodies: the support yoke and the central body. The support yoke is constrained to the ground via a cylindrical joint, allowing rotation around a single vertical axis (yaw). The central body is attached to the support yoke through a planar joint, allowing rotation about a horizontal axis (pitch).

The following assumptions are made to simplify the model:

- The support yoke is both statically balanced around its axis of rotation, i.e. its center of mass is on the axis of rotation.

- The center of mass of the central body is misaligned relative to its pitch rotation axis.
- The kinetic energy contribution from the propellers (fans) themselves is neglected.

The model includes the following non-controllable (dissipative) forces:

- Static friction at the base of the support yoke.
- Damping forces in the yaw and pitch rotations, attributed primarily to aerodynamic interactions (e.g., wind resistance).

Additionally, the system experiences controllable forces, specifically:

- Thrust generated by the propellers, directed along their axes of rotation.
- Coupling torque arising from electromagnetic interactions between the rotor and stator of the DC motors.

2.1. Reference system

To describe the motion, we introduce three distinct reference frames, each defined by an origin and three orthonormal unit vectors forming right-handed coordinate systems:

- **Global Frame** ($O_g; x_G, y_G, z_G$): The origin O_g is positioned at the base of the support yoke. The z^G -axis is oriented vertically upwards, the x^G -axis is horizontal, and the y^G -axis is defined to complete a right-handed coordinate system.
- **Local Frame of the Support Yoke** ($O_1; x_1, y_1, z_1$): The origin O_1 coincides with the center of mass of the support yoke. The z_1 -axis aligns with the global vertical axis z^G . Axes x_1 and y_1 lie in the horizontal plane orthogonal to z_1 and are attached to the yoke, forming an orthonormal right-handed coordinate frame.
- **Local Frame of the Central Body** ($O_2; x_2, y_2, z_2$): The origin O_2 is located at the planar joint connecting the central body to the support yoke. The x_2 -axis is aligned along the central body's main axis. The z_2 -axis is orthogonal to the x_2 -axis and lies in the plane defined by the global axes x^G and z^G . The y_2 -axis is selected to complete the right-handed coordinate system.

As previously stated, the model considers the 2 DOF (Degrees of Freedom) operational mode. Here we precisely define these two degrees as:

- **Yaw angle** ψ : The angle between the global axis x_G and the local axis x_1 , measured as a rotation around the z_G axis. Positive direction is defined as counter-clockwise rotation.
- **Pitch angle** θ : The angle from the axis x_1 to the axis x_2 , measured as a rotation around the local axis y_2 . Positive direction is defined as clockwise rotation.

2.2. Lagrange equations

To derive the equations of motion, we employ the Lagrange formalism. For the two degrees of freedom considered (yaw angle ψ and pitch angle θ), the Lagrange equations read:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\psi}} \right) - \frac{\partial L}{\partial \psi} = \tau_\psi$$

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}} \right) - \frac{\partial L}{\partial \theta} = \tau_\theta$$

Here, L represents the Lagrangian defined as $L = T_{tot} - U_{tot}$, where T_{tot} is the kinetic energy associated with the entire system, and U_{tot} is the potential energy. The terms τ_ψ and τ_θ represent the generalized non-conservative forces corresponding to the yaw and pitch degrees of freedom, respectively.

In the subsequent sections, we first compute the Lagrangian and then derive the generalized forces.

2.2.1. Computing the Lagrangian

The kinetic energy of a rigid body about its center of mass consists of two components: the rotational kinetic energy, defined by its angular velocity $\boldsymbol{\omega}$ and the inertia tensor about the center of mass, and the translational kinetic energy, associated with the velocity of the body's center of mass.

The kinetic energy T_{yoke} only includes a rotational component, since the support yoke is assumed statically balanced:

$$T_{\text{yoke}} = \frac{1}{2} \boldsymbol{\omega}_{\text{yoke}}^\top \mathbf{J}_{\text{yoke}} \boldsymbol{\omega}_{\text{yoke}}$$

The inertia tensor and the angular velocity are expressed in the local reference frame (O_1, x_1, y_1, z_1). Although the inertia tensor may generally contain nonzero off-diagonal and diagonal terms, only the component J_{z_1} contributes to the kinetic energy due to the form of $\boldsymbol{\omega}_{\text{yoke}}$. Both quantities are given below:

$$\mathbf{J}_{\text{yoke}} = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & J_{z_1} \end{bmatrix}, \quad \boldsymbol{\omega}_{\text{yoke}} = \begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix} \quad (1)$$

The kinetic energy of the central body T_{body} includes both rotational and translational components:

$$T_{\text{body}} = \frac{1}{2} \boldsymbol{\omega}_{\text{body}}^\top \mathbf{J}_{\text{body}} \boldsymbol{\omega}_{\text{body}} + \frac{1}{2} \mathbf{v}_{G_2}^\top M_{\text{body}} \mathbf{v}_{G_2}$$

These quantities are expressed with respect to the local reference frame of the central body (O_2, x_2, y_2, z_2) . Moreover, it is assumed that this frame is formed by principal axes of inertia, resulting in a diagonal inertia matrix. They are given by:

$$\mathbf{J}_{\text{body}} = \begin{bmatrix} J_{x_2} & 0 & 0 \\ 0 & J_{y_2} & 0 \\ 0 & 0 & J_{z_2} \end{bmatrix}, \quad \boldsymbol{\omega}_{\text{body}} = \begin{bmatrix} \dot{\psi} \sin \theta \\ \dot{\theta} \\ \dot{\psi} \cos \theta \end{bmatrix}, \quad \mathbf{v}_{G_2} = \begin{bmatrix} D_m \dot{\theta} \\ D_m \dot{\psi} \sin \theta \\ 0 \end{bmatrix} \quad (2)$$

The potential energy U is given solely by the gravitational contribution of the central body:

$$U = -M_{\text{body}} g D_m \cos \theta \quad (3)$$

Here, g denotes the acceleration due to gravity, and D_m represents the distance from the planar joint to the center of mass of the body.

We now have all the ingredients to write the Lagrangian of the system, which takes the form:

$$L = \frac{1}{2} (J_{z_1} \dot{\psi}^2 + J_{x_2} \dot{\psi}^2 \sin^2 \theta + J_{z_2} \dot{\psi}^2 \cos^2 \theta + J_{y_2} \dot{\theta}^2 + M_{\text{body}} D_m^2 (\dot{\theta}^2 + \dot{\psi}^2 \sin^2 \theta)) + M_{\text{body}} g D_m \cos \theta \quad (4)$$

2.2.2. Computing the Generalized Forces

The generalized forces are computed by considering the differential of the virtual work performed by the forces acting on the system. The virtual work of a force is defined as:

$$\delta W = \mathbf{F}^\top \delta \mathbf{P},$$

where $\delta \mathbf{P}$ denotes the virtual displacement of the point $\mathbf{P}$, which is the point of application of the force. Similarly, the virtual work done by a moment is:

$$\delta W = \mathbf{M}^\top \delta \boldsymbol{\omega}.$$

To compute the generalized forces, the expressions for virtual work must be projected onto the virtual variations of the generalized coordinates, which in our case are θ and ψ . This yields a representation of δW in terms of $\delta \theta$ and $\delta \psi$, from which the corresponding torques can be identified by comparison with the general form:

$$\delta W = \tau_\theta \delta \theta + \tau_\psi \delta \psi.$$

Below, we detail the forces acting on the system and compute their contributions to the generalized forces. We show the first force in more detail, as subsequent forces follow similar procedures.

Thrust Forces of the Fans Due to aerodynamic interaction, each fan produces a thrust force along its axis. The horizontal fan thrust F_h acts along the z_2 -axis, while the vertical fan thrust F_v acts along the y_2 -axis. Denoting $\mathbf{P}_h$ and $\mathbf{P}_v$ as the points of application for these forces the total virtual work due to thrust is:

$$\delta W_{\text{thrust}} = \mathbf{F}_h^\top \delta \mathbf{P}_h + \mathbf{F}_v^\top \delta \mathbf{P}_v.$$

Expressing these vectors in the local reference frame (x_2, y_2, z_2) , we have:

$$\delta \mathbf{P}_h = \begin{bmatrix} 0 \\ -L_m \cos \theta \delta \psi \\ L_m \delta \theta \end{bmatrix}, \quad \delta \mathbf{P}_v = \begin{bmatrix} 0 \\ L_m \cos \theta \delta \psi \\ -L_m \delta \theta \end{bmatrix}, \quad \mathbf{F}_h = \begin{bmatrix} 0 \\ 0 \\ F_h \end{bmatrix}, \quad \mathbf{F}_v = \begin{bmatrix} 0 \\ F_v \\ 0 \end{bmatrix}$$

where L_m represents the distance from the planar joint to the motor axis. The resulting virtual work is:

$$\delta W_{\text{thrust}} = F_h L_m \delta \theta + F_v L_m \cos \theta \delta \psi,$$

from which we identify the contributions to generalized forces as:

$$\tau_\psi^{\text{thrust}} = F_v L_m \cos \theta, \quad \tau_\theta^{\text{thrust}} = F_h L_m.$$

Coupling Torque of the Fan Motors The coupling torque results from electromagnetic interactions between the stator (fixed to the mechanical structure) and the rotor. Thus, these torques align with the respective motor axes. Expressing these vectors in the local frame (x_2, y_2, z_2) :

$$\mathbf{Q}_h = \begin{bmatrix} 0 \\ 0 \\ -Q_h \end{bmatrix}, \quad \mathbf{Q}_v = \begin{bmatrix} 0 \\ -Q_v \\ 0 \end{bmatrix}$$

Using the angular velocities from equations (2) and (1), we obtain:

$$\tau_\psi^{\text{coupling}} = -Q_h \cos \theta, \quad \tau_\theta^{\text{coupling}} = -Q_v.$$

Friction Force and Damping The friction at the base is modeled as a static torque along the z_1 -axis. Damping forces, proportional to the yaw and pitch velocities, are directly generalized forces. Thus, their contributions are:

$$\tau_\psi^{\text{fd}} = -\text{sign}(\dot{\psi}) \mu - \beta_\psi, \quad \tau_\theta^{\text{fd}} = -\beta_\theta.$$

Total Generalized Forces By summing the individual contributions from thrust, coupling torques, and friction/damping, the total generalized forces take the form:

$$\tau_\psi = \tau_\psi^{\text{thrust}} + \tau_\psi^{\text{coupling}} + \tau_\psi^{\text{fd}} = F_v L_m \cos \theta - Q_h \cos \theta - \text{sign}(\dot{\psi}) \mu - \beta_\psi,$$

$$\tau_\theta = \tau_\theta^{\text{thrust}} + \tau_\theta^{\text{coupling}} + \tau_\theta^{\text{fd}} = F_h L_m - Q_v - \beta_\theta.$$

2.3. Equations of motion

With all components now defined, we can apply the Lagrange formalism to derive the equations of motion. This results in the following dynamic model:

Yaw-motion:

$$\ddot{\psi}(J_{z_1} + J_{x_2} \sin^2 \theta + J_{z_2} \cos^2 \theta + M_{\text{body}} D_m^2 \sin^2 \theta) + \tag{5}$$

$$+ \dot{\psi} \dot{\theta} \sin 2\theta [(J_{x_2} - J_{z_2}) + M_{\text{body}} D_m^2] + \dot{\psi} \beta_\psi$$

$$= F_v \cos \theta L_m - Q_h \cos \theta - \text{sign}(\dot{\psi}) \mu \tag{6}$$

Pitch-motion:

$$\ddot{\theta}(J_{y_2} + M_{\text{body}} D_m^2) - \frac{1}{2} \dot{\psi}^2 \sin 2\theta [(J_{x_2} - J_{z_2}) + M_{\text{body}} D_m^2] + \dot{\theta} \beta_\theta + M_{\text{body}} g D_m \sin \theta = F_h L_m - Q_v \tag{7}$$

3. Propeller Dynamics and Control

The propeller is an essential component of the system, providing the thrust required to influence the dynamics of the Quanser AERO.

In this section, the mathematical model used to describe its behavior is introduced, followed by the model adopted for control design and the corresponding parameter identification. Finally, the implementation of a PI controller with anti-windup compensation is presented, testing different cutoff frequency targets.

3.1. Propeller Model

Electrical Dynamics The electrical dynamics of a permanent magnet DC motor are described by the following circuit equation:

$$v = R_a i + L_a \frac{di}{dt} + e(\Omega_m)$$

where R_a and L_a denote the armature resistance and inductance, respectively, and $e(\Omega_m)$ is the back-electromotive force (back-EMF), which is proportional to the mechanical angular velocity of the rotor Ω_m :

$$e(\Omega_m) = K_e \Omega_m$$

with K_e being the back-EMF constant.

Mechanical Dynamics The mechanical dynamics involve balancing the electric torque generated by the motor with the aerodynamic torque and damping effects acting on the rotor. The resulting equation of motion is:

$$J\dot{\Omega}_m + C_{fan}\Omega_m + \gamma \Omega_m^2 \text{sign}(\Omega_m) = K_t i$$

where J is the rotor's moment of inertia, C_{fan} represents the damping coefficient, γ characterizes the aerodynamic torque proportional to the square of the rotor speed, and K_t is the motor torque constant relating the electrical current i to the produced torque.

3.2. Propeller Control

The control design leverages the fact that the electrical dynamics of the motor are significantly faster than the mechanical dynamics. Moreover, since a sufficiently fast current sensor is unavailable, direct current control is not feasible. Thus, a single-loop control strategy is adopted, focusing on controlling the mechanical speed of the propeller.

The available control input is the voltage reference supplied to the power converter, which is assumed to perfectly track the input voltage. Consequently, to proceed with the controller design, we must derive a transfer function that relates the applied voltage to the rotor's angular velocity. By combining the electrical and mechanical dynamics previously introduced, we obtain:

$$\Omega_m(s) = \frac{\mu_{prop}}{(1 + \tau_{prop}s)} V, \quad \text{with} \quad \tau_{prop} = \frac{J_{motor}}{C_{fan}}, \quad \mu_{prop} = \frac{K_t}{C_{fan} R} \quad (8)$$

Note that the transfer function is not from the current but from the voltage, therefore the identified parameters will not match directly any physical property of the system, but rather a combination of them. The nonlinear term proportional to Ω_m^2 is neglected in the control design, as its contribution in transient is relatively small, and it is compensated by the integrator at steady state. In identification a square term will be included to improve the estimation of the gain, but it will not directly match the physical torque coefficient, which will be estimated later.

A PI controller is implemented with the goal of achieving a desired cutoff frequency and a phase margin of approximately 90° . The controller is defined as:

$$\mathbf{C}(s) = K_p + \frac{K_I}{s}$$

The controller parameters are determined using the following expressions:

$$\begin{aligned} K_p &= \frac{\omega_c \tau_{prop}}{\mu_{prop}}, \\ K_I &= \frac{K_p}{\tau_{prop}}, \end{aligned} \quad (9)$$

where ω_c is the desired cutoff frequency, τ_{prop} is the system time constant, and μ_{prop} is the system gain. Finally, to address actuator saturation we opted to use the clamping method available in MATLAB.

3.3. Identification

Since the goal was only to tune the PID, and not to identify the physical parameters, the identification procedure was focused only on finding μ_{prop} , τ_{prop} and γ , disregarding the real parameters that compose them. To find such parameters we gathered a dataset of 1 Volt steps, and then set up an optimization problem to compare simulation and real data:

$$\begin{aligned} \min_{\tau_{prop}, \mu_{prop}, \gamma} \quad & \|\omega_{sim}(V; \tau_{prop}, \mu_{prop}, \gamma) - \omega_{meas}\|_2^2 \\ \text{subject to} \quad & \omega_{sim}^i = \mathcal{M}(V^i, \omega_{sim}^{i-1}, \tau_{prop}, \mu_{prop}), \\ & \tau_{prop} > 0, \\ & \mu_{prop} > 0 \\ & \gamma > 0 \end{aligned} \quad (10)$$

In practice the optimization problem was solved in two separate instances, estimating first γ and μ_{prop} disregarding the transient, and then τ_{prop} to estimate the transient. Estimating all three at the same time was not feasible because on the entire dataset τ_{prop} had a very small weight in the cost function, affecting only the short transients. The model was simulated using Simulink, and the optimization problem was solved through fmincon.

As we can see the model works very well for most values of Omega positive and negative, only at very low voltages there is some nonlinear uncaught behavior, but it's clearly neglectable.

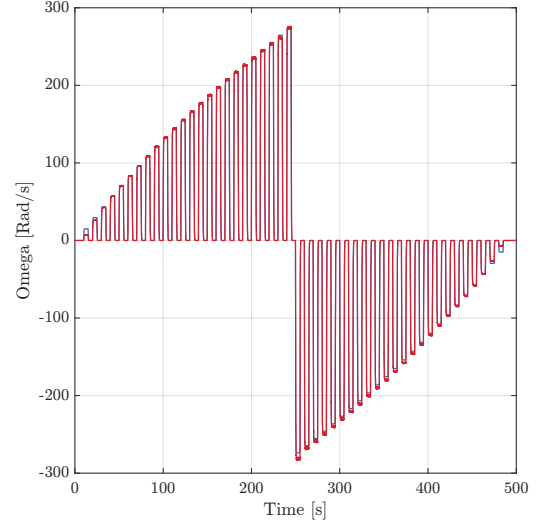
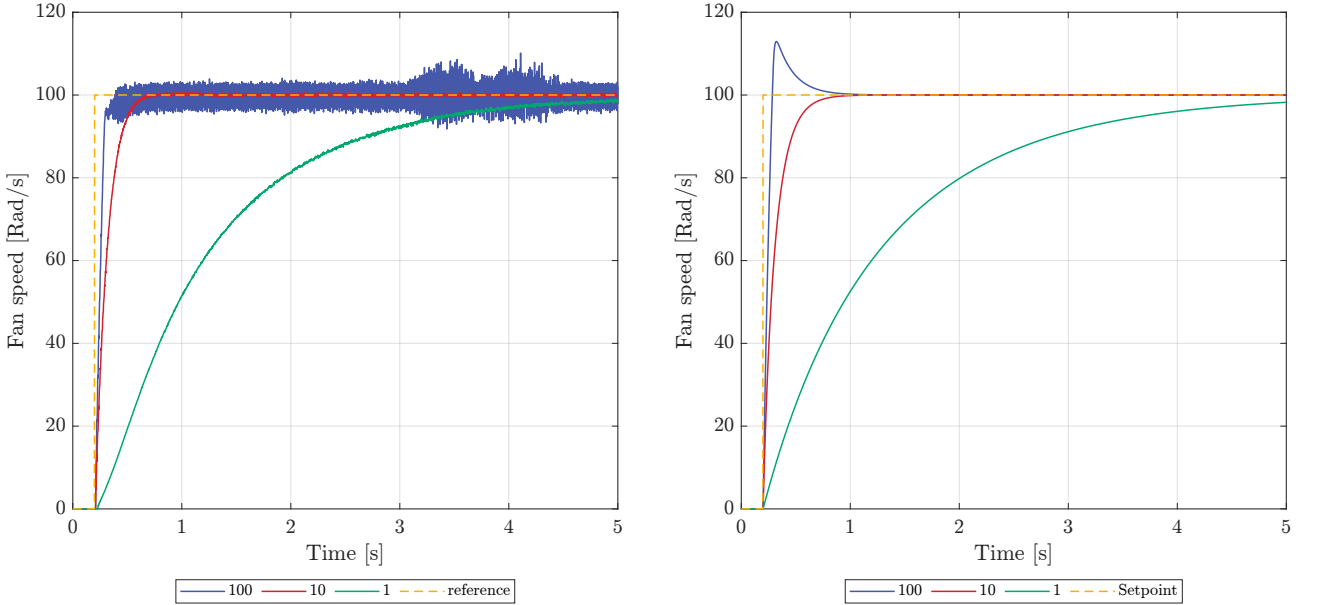


Figure 2: Comparison between simulated and measured omega

3.4. Implementation results

In this section, we present results obtained using different cutoff frequencies. The objective is to assess how aggressively the propeller dynamics can be driven while still maintaining satisfactory performance. Experimentally we noticed that compensation of the square term was not necessary, our explanation for that is that since it mostly affects steady state it can be fully substituted by the integrator.

Increasing the control bandwidth of the propeller enables faster actuation, which is essential for achieving higher-performance control of pitch and yaw in future developments—without requiring the propeller dynamics to be explicitly considered in the overall system design.



(a) Responses with different PIDs, tuned to get a cuttng frequency of 100, 10, 1 Hz

(b) Simulated Ω_m control with the same controllers

Figure 3: Comparison between real and simulated Ω_m control using different PID controllers.

It's clear that the control at 10Hz is the best one, with the correct compromise between speed and noise rejection, given that the 100Hz controller made audible noise. For this reason from now on all PIDs will be tuned to be at least a decade slower. This is not a big limitation for the 1DOF controller, but it is for the 2DOF PID controllers, that have to follow a very fast changing signal.

In simulations the results are very similar up to the 10Hz controller, the physical limitations of the faster dynamics are not modeled, but they are of no interest for our scope.

4. Modelling the System in 2 DOF

4.1. State Space Representation of Mechanical System

Starting from the model defined in [Sec. 2] we can construct the state-space representation by defining the following state and input variables:

$$\begin{aligned} x_1 &= \theta, \text{ pitch angle} \\ x_2 &= \dot{\theta}, \text{ rate of change of pitch angle (pitch speed)} \\ x_3 &= \psi, \text{ yaw angle} \\ x_4 &= \dot{\psi}, \text{ rate of change of yaw angle (yaw speed)} \\ u_1 &= \Omega_0^2, \text{ squared speed of Motor 0} \\ u_2 &= \Omega_1^2, \text{ squared speed of Motor 1} \end{aligned}$$

The model thus becomes:

$$\begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= \frac{1}{J_\theta} [\tau_\theta - \beta_\theta x_2 - \text{coup}_\theta - M_{body} g D_m \sin(x_1)] \\ \dot{x}_3 &= x_4 \\ \dot{x}_4 &= \frac{1}{J_\psi} [\tau_\psi - \beta_\psi x_4 - \text{coup}_\psi - \mu] \end{aligned}$$

where

$$\begin{aligned} \tau_\theta &= \beta u_1 L_m + c_{Q1} u_2 \\ \tau_\psi &= (\beta u_2 L_m + c_{Q0} u_1) \cos(x_1) \\ J_\theta &= J_{y2} + M_{body} D_m^2 \\ J_\psi &= J_{z1} + J_{x2} \sin^2(x_1) + J_{z2} \cos^2(x_1) + M_{body} D_m^2 \sin^2(x_1) \\ \text{coup}_\theta &= -0.5(J_{x2} - J_{z2} + M_{body} D_m^2) \sin(2x_1) x_4^2 \\ \text{coup}_\psi &= (J_{x2} - J_{z2} + M_{body} D_m^2) \sin(2x_1) x_4 x_2 \\ \mu &: \text{could represent either static or dynamic friction} \end{aligned}$$

4.2. State Space Representation of the Full System

In this subsection the dynamics of the propeller are incorporated into the system model so that the whole system is simulated by providing the input voltages. The mathematical formulation is shown in the next subsection. Specifically, two new state variables are introduced and the input variables change:

$$\begin{aligned} x_5 &= \Omega_0, \text{ speed of Motor 0} \\ x_6 &= \Omega_1, \text{ speed of Motor 1} \\ u_1 &= V_0, \text{ voltage applied to Motor 0} \\ u_2 &= V_1, \text{ voltage applied to Motor 1} \end{aligned}$$

The control torques, τ_θ and τ_ψ , are also updated to directly depend on the squared motor speeds and their sign. The extended state-space model becomes:

$$\begin{aligned}
\dot{x}_1 &= x_2 \\
\dot{x}_2 &= \frac{1}{J_\theta} [\tau_\theta - \beta_\theta x_2 - \text{coup}_\theta - M_{body} g D_m \sin(x_1)] \\
\dot{x}_3 &= x_4 \\
\dot{x}_4 &= \frac{1}{J_\psi} [\tau_\psi - \beta_\psi x_4 - \text{coup}_\psi - \text{sign}(x_4) \mu] \\
\dot{x}_5 &= \frac{\mu_{\text{prop}}}{\tau_{\text{prop}}} u_1 - \frac{\mu_{\text{prop}}}{\tau_{\text{prop}}} \gamma \text{sign}(x_5) x_5^2 - \frac{1}{\tau_{\text{prop}}} x_5 \\
\dot{x}_6 &= \frac{\mu_{\text{prop}}}{\tau_{\text{prop}}} u_2 - \frac{\mu_{\text{prop}}}{\tau_{\text{prop}}} \gamma \text{sign}(x_6) x_6^2 - \frac{1}{\tau_{\text{prop}}} x_6
\end{aligned}$$

where

$$\begin{aligned}
\tau_\theta &= \beta x_5^2 L_m \text{sign}(x_5) + c_{Q1} x_6^2 \text{sign}(x_6) \\
\tau_\psi &= (\beta x_6^2 L_m \text{sign}(x_6) + c_{Q0} x_5^2 \text{sign}(x_5)) \cos(x_1) \\
J_\theta &= J_{y2} + M_{body} D_m^2 \\
J_\psi &= J_{z1} + J_{x2} \sin^2(x_1) + J_{z2} \cos^2(x_1) + M_{body} D_m^2 \sin^2(x_1) \\
\text{coup}_\theta &= -0.5(J_{x2} - J_{z2} + M_{body} D_m^2) \sin(2x_1) x_4^2 \\
\text{coup}_\psi &= (J_{x2} - J_{z2} + M_{body} D_m^2) \sin(2x_1) x_4 x_2
\end{aligned}$$

4.3. Linearisation of the Full System

he system can have infinite equilibrium points; therefore, the linearisation is performed on the *symbolic system* around a chosen equilibrium point $\mathbf{x}_e$, with corresponding input $\mathbf{u}_e$.

To linearise the nonlinear system:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}),$$

we perform a first-order Taylor expansion around $(\mathbf{x}_e, \mathbf{u}_e)$:

$$\delta \dot{\mathbf{x}} = A \delta \mathbf{x} + B \delta \mathbf{u},$$

where

$$A = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{(\mathbf{x}_e, \mathbf{u}_e)}, \quad B = \left. \frac{\partial f}{\partial \mathbf{u}} \right|_{(\mathbf{x}_e, \mathbf{u}_e)},$$

and

$$\delta \mathbf{x} = \mathbf{x} - \mathbf{x}_e, \quad \delta \mathbf{u} = \mathbf{u} - \mathbf{u}_e.$$

The matrices A and B represent the *Jacobian matrices* of the system evaluated at the equilibrium point.

After performing the symbolic linearisation as described, we obtain the system matrices A_{sym} and B_{sym} that describe the local behavior of the nonlinear system around the chosen equilibrium point. Specifically, we have:

$$\dot{\mathbf{x}} = A_{\text{sym}} \mathbf{x} + B_{\text{sym}} \mathbf{u},$$

where $\mathbf{x} = [\theta, \dot{\theta}, \psi, \dot{\psi}, \Omega_0, \Omega_1]^\top$ and $\mathbf{u} = [V_0, V_1]^\top$.

The matrices are:

$$A_{\text{sym}} = \begin{bmatrix}
0 & 1 & 0 \\
\frac{-M_{body} g D_m \cos(x_1) + \partial_{x_1} \text{coup}_\theta}{J_\theta} & \frac{-\beta_\theta}{J_\theta} & 0 \\
0 & 0 & 0 \\
\frac{\partial_{x_1} \tau_\psi - \partial_{x_1} \text{coup}_\psi}{J_\psi} & \frac{-\partial_{x_2} \text{coup}_\psi}{J_\psi} & \dots \\
0 & 0 & 0 \\
0 & 0 & 0 \\
0 & 0 & 0 \\
\frac{\partial_{x_4} \text{coup}_\theta}{J_\theta} & \frac{\partial_{x_5} \tau_\theta}{J_\theta} & \frac{\partial_{x_6} \tau_\theta}{J_\theta} \\
1 & 0 & 0 \\
\frac{-\beta_\psi - \partial_{x_4} \mu \text{sign}(x_4)}{J_\psi} & \frac{\partial_{x_5} \tau_\psi}{J_\psi} & \frac{\partial_{x_6} \tau_\psi}{J_\psi} \\
0 & -\frac{1}{\tau_{\text{prop}}} - \frac{2\gamma \mu_{\text{prop}} x_5 \text{sign}(x_5)}{\tau_{\text{prop}}} & 0 \\
0 & 0 & -\frac{1}{\tau_{\text{prop}}} - \frac{2\gamma \mu_{\text{prop}} x_6 \text{sign}(x_6)}{\tau_{\text{prop}}}
\end{bmatrix}$$

$$B_{\text{sym}} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ \frac{\mu_{\text{prop}}}{\tau_{\text{prop}}} & 0 \\ 0 & \frac{\mu_{\text{prop}}}{\tau_{\text{prop}}} \end{bmatrix}$$

To find the equilibrium points, we impose $\dot{\mathbf{x}} = 0$ and solve the resulting system. Since the system consists of six equations in eight variables, the solution set describes a two-dimensional manifold $\mathcal{M} \subset \mathbb{R}^8$. We therefore fix arbitrary equilibrium values for x_1 and x_3 , and solve for the remaining variables. As the system is too complex for analytical inversion, we use the numerical solver `fsolve()` in MATLAB to obtain the equilibrium solutions.

4.3.1. Linearisation of Mechanical System

If, instead, the model of the mechanical system alone, taking the motor speeds as inputs was to be linearised following the same procedure, the symbolic matrices would be:

$$A_{\text{sym}} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ \frac{-M_{\text{body}} g D_m \cos(x_1) + \partial_{x_1} \text{coup}_\theta}{J_\theta} & \frac{-\beta_\theta}{J_\theta} & 0 & \frac{\partial_{x_4} \text{coup}_\theta}{J_\theta} \\ 0 & 0 & 0 & 1 \\ \frac{\partial_{x_1} \tau_\psi - \partial_{x_1} \text{coup}_\psi}{J_\psi} & \frac{-\partial_{x_2} \text{coup}_\psi}{J_\psi} & 0 & \frac{-\beta_\psi - \partial_{x_4} \mu \text{sign}(x_4)}{J_\psi} \end{bmatrix}$$

$$B_{\text{sym}} = \begin{bmatrix} 0 & 0 \\ \frac{\beta L_m}{J_\theta} & \frac{c_{Q1}}{J_\theta} \\ 0 & 0 \\ \frac{\beta L_m \cos x_1}{J_\psi} & \frac{c_{Q0} \cos x_1}{J_\psi} \end{bmatrix}$$

To find the equilibrium points, we again impose $\dot{\mathbf{x}} = 0$ and solve the resulting system.

5. 1 DOF VTOL Modeling and Control

In this section, we analyze and control the Quanser AERO system operating in the 1DOF VTOL configuration. In this mode, the system is constrained such that rotation around the support yoke (yaw motion) is locked, and both propellers are oriented horizontally. This simplifies the system to allow motion only in the pitch direction. The objective is to evaluate the performance of different control schemes in achieving both stability around the equilibrium point and trajectory tracking. To this end, we begin by deriving the dynamical model corresponding to this reduced configuration. The model is obtained by simplifying the previously derived 2 DOF model; this process is detailed in the next subsection.

Following the modeling, we perform parameter identification to obtain the physical parameters relevant to the 1 DOF setup. Finally, we implement and compare three control strategies: classical PID control, Fuzzy-PID control, and Model Predictive Control (MPC). For each approach, the structure, tuning methodology, and experimental results are discussed.

5.1. 1 DOF Model

In the 1 DOF VTOL mode of operation, the only active degree of freedom is the pitch angle. To derive the corresponding model, we start from the pitch dynamics equation of the full 2DOF system, equation 7. The simplification process consists of:

- Setting $\dot{\psi} = 0$, since yaw rotation is disabled;
- Removing the torque term Q_v , which is associated with the vertical rotor;
- Replacing it with a thrust force contribution, as both rotors are now oriented horizontally and contribute symmetrically to the pitch motion.

With these assumptions, the pitch dynamics simplify to the following form:

$$(F_0 - F_1)L_m - \beta_\theta \dot{\theta} - M_{\text{body}} g d \sin(\theta) = J_p \ddot{\theta} \quad (11)$$

Note that the forces are now just identified by the motors, since both work to accomplish the same movement. This is possible by simply giving the same speed reference to the inner loop, but with opposite signs.

5.1.1. Linearisation

The linearisation of the mechanical model of 1DOF is easily found from the one found in 4.3. By substituting $\dot{\psi} = 0$ and $coup_{\theta} = 0$ and cutting the system we get:

$$A_{\text{sym}} = \begin{bmatrix} 0 & 1 \\ \frac{-M_{\text{body}}gD_m \cos(x_1)}{J_{\theta}} & \frac{-\beta_{\theta}}{J_{\theta}} \end{bmatrix}$$

$$B_{\text{sym}} = \begin{bmatrix} 0 \\ \frac{\beta L_m}{J_{\theta}} \end{bmatrix}$$

5.2. Parameter Identification

5.2.1. Force Coefficient

The thrust force applied to the fan is assumed to be proportional to the square of the fan speed and can be modeled as

$$F = \text{sign}(\Omega_m) \cdot \beta \Omega_m^2 \quad (12)$$

where β is the force coefficient to be identified. This is a simplified approximation; the net thrust force applied to the fan is easily affected by movements and wind around the fan, and it is therefore unrealistic to capture the exact model. It is important to note that, for simplicity, it is assumed that the magnitudes of force due to positive and negative angular velocity are equal and that the motor models of the two propellers are identical.

To identify β , a series of voltage steps was applied to Motor 0 and the angular speed and pitch angle were measured. Each value of voltage was applied for a long enough time for the initial transient of the system to settle and the steady state to be reached. The measured pitch angle θ_{eq}^i is then recorded and used to find the force applied at the equilibrium position F_{eq} according to the model pitch dynamics with constant pitch and no force applied to Motor 1.

$$F_{\text{eq}}^i = \frac{M_{\text{body}} g D_m \sin(\theta_{\text{eq}}^i)}{L_m} \quad (13)$$

In practice, the pitch angle measurements did not settle to an equilibrium position after the initial transient, but rather oscillated around a certain value, likely due to disturbances from the environment around the system. To mitigate this issue, the average value of the pitch angle after every step of voltage was calculated using MATLAB to approximate the equilibrium angle θ_{eq}^i . The same process was performed on the motor speed measurements to obtain Ω_{eq}^i to acquire a relationship between the motor speed and the thrust force. Note that the motor speed control loop dynamics are fast enough to be neglected, and the change in speed with respect to voltage can be viewed as instantaneous. Now that the force speed relationship model has been calculated, an optimization problem is formalized to find the optimal β using MATLAB's `fmincon()`. The function takes as input the equilibrium speeds and minimizes the objective function over β .

$$\min_{\beta} \|F_{\text{eq}} - F_0\|_2^2$$

subject to $F_0^i = \text{sign}(\Omega_{\text{eq}}^i) \cdot \beta (\Omega_{\text{eq}}^i)^2$

(14)

The initial assumption that the force is proportional to the square of the speed can be validated by plotting and inspecting the measured and computed equilibrium forces against the equilibrium speeds as shown below.

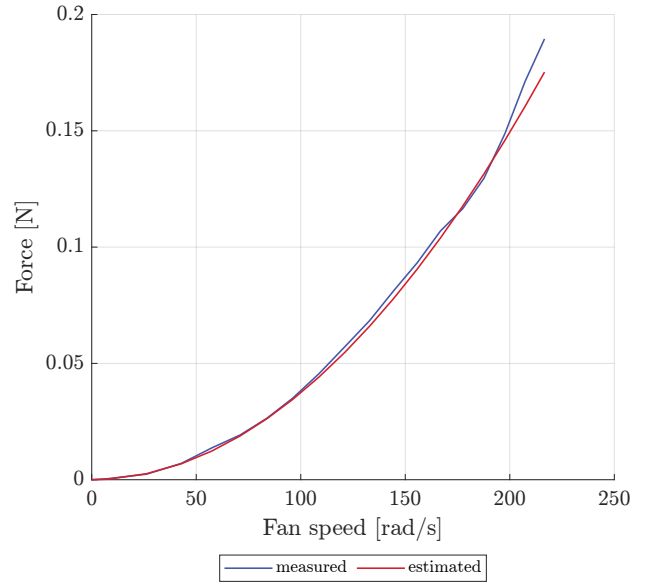


Figure 4: Comparison between Force-Speed relationship from measured equilibria and estimated coefficient

5.2.2. Damping

Once the force was known the dynamic response to different voltage steps was gathered, and from the corresponding oscillations we gathered an initial guess of the damping. Then from there an optimisation problem was set up to improve the value to the optimal one.

$$\begin{aligned} \min_C \quad & \|\theta_{\text{sim}}(V; \beta_\theta) - \theta_{\text{meas}}\|_2^2 \\ \text{subject to} \quad & \theta_{\text{sim}}^i = \mathcal{M}(V^i, \theta_{\text{sim}}^{i-1}, \Omega_m^i, \beta_\theta) \\ & \beta_\theta > 0 \end{aligned} \quad (15)$$

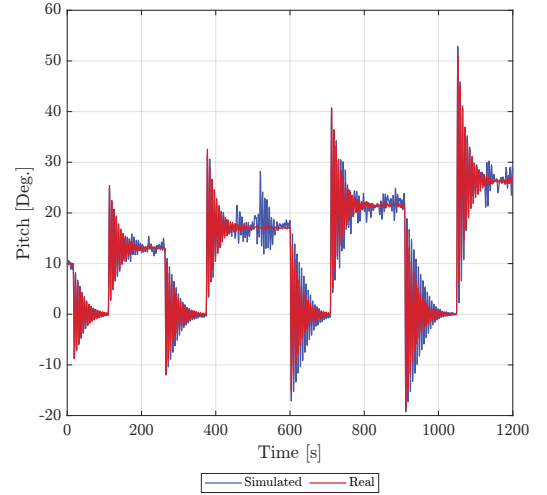


Figure 5: Comparison between the simulation and real data using identified parameters

5.3. PID Control

PID control is the starter control strategy in most control projects given it's relative simplicity yet very robust and effective performance. The main control scheme is shown in Figure 6 below.

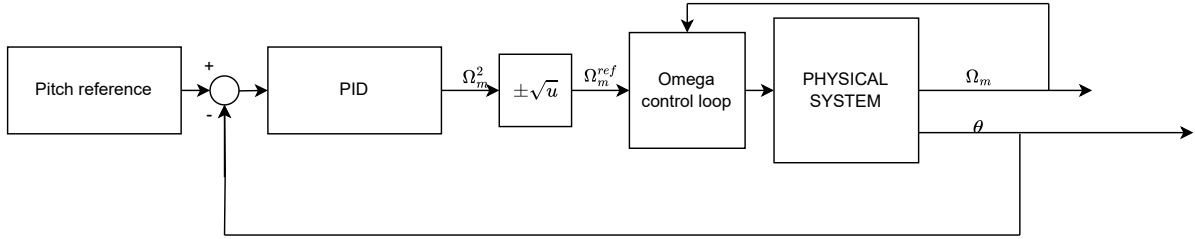


Figure 6: Control scheme

In this implementation, for the 1 DOF case, the controller tuning was performed over multiple stages. Firstly, analytical tuning based on a transfer function model of the pitch, was carried out. Then, after experimenting on the real system starting from the analytical PID, through several iterations the final PD-like controller was reached. For the analytical initial tuning Equation (11) was used to derive the transfer function from Ω_m^2 to pitch:

$$G_{pitch}(s) = \frac{L_m \beta}{\beta_\theta} \frac{1}{s(\tau_p s + 1)} \quad \text{with } \tau_p = \frac{J_p}{\beta_\theta}$$

The objective of the initial PID tuning was to cancel out the pole $(\tau_p s + 1)$, substitute it with $(\tau_{sett} s + 1)$ where $\tau_{sett} = 0.05$ s.

$$C(s) = \mu \frac{(1 + 10s)(1 + \tau_p s)}{s(1 + \tau_{sett} s)} \quad \text{with } \mu = \frac{\beta_\theta}{10L_m \beta}$$

By doing so we get a 81.4° phase margin with an easily tuneable cutting frequency.

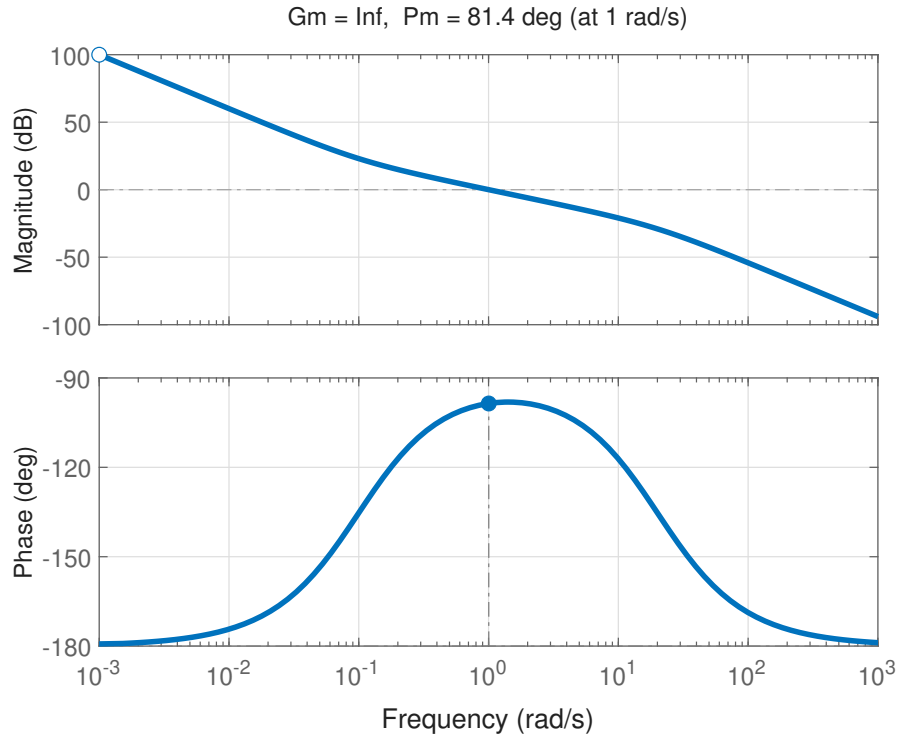


Figure 7: Bode plot of analytical PID

Testing it on the real system, the control response was noticeably slow, probably due to the exaggerated phase margin creating an overly cautious controller.

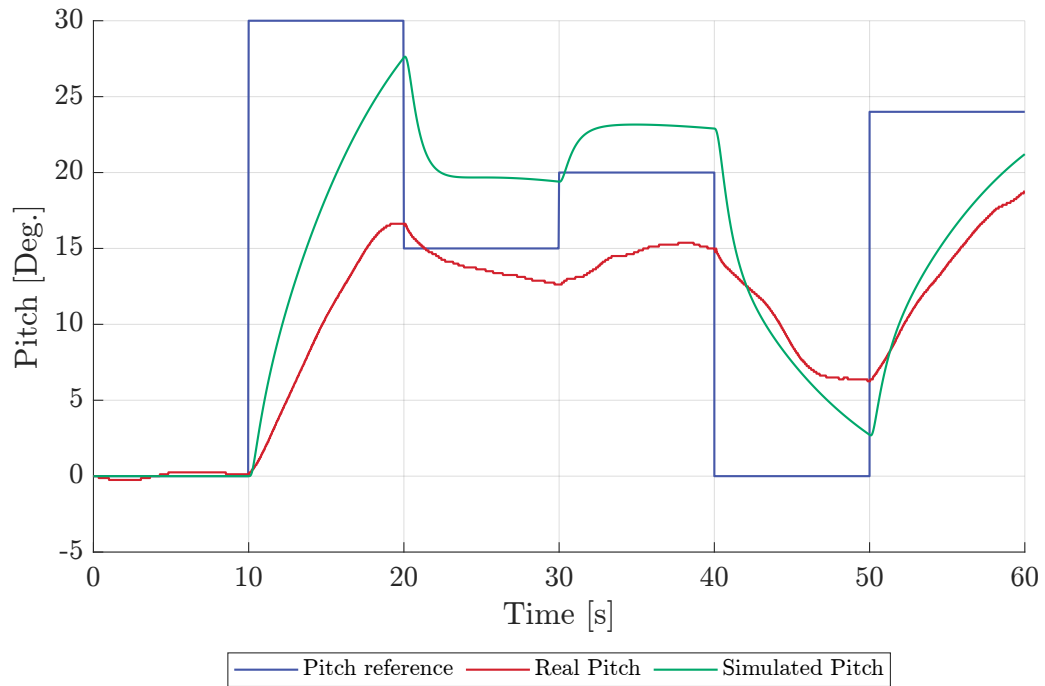


Figure 8: Step response comparison of the real and simulated system to control pitch in 1 DOF case with analytic PID

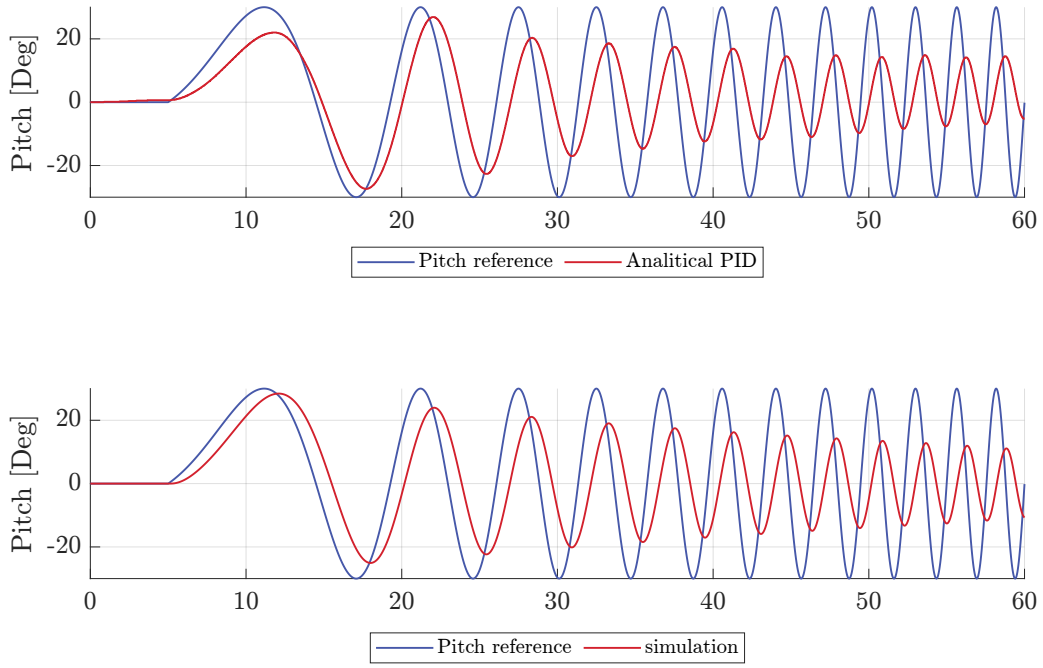


Figure 9: Comparison of the real and simulated system to control pitch in 1 DOF case with analytic PID, following a variable-frequency sine trajectory

After the initial attempt the PID controller was tuned manually. Based on empirical results, it was concluded that the best system was a PID with very low integral action so to avoid lower phase margins caused by $\frac{1}{s^2}$ in the open loop transfer function but keep some robustness for steady state tracking. From now on we refer to it as a PD controller, since the integral part is orders of magnitudes lower, and only has a noticeable effect over a longer time span. The bode plot is shown in Figure 10, and the system responses to the same reference trajectories as before are shown in Figures 11 and 13.

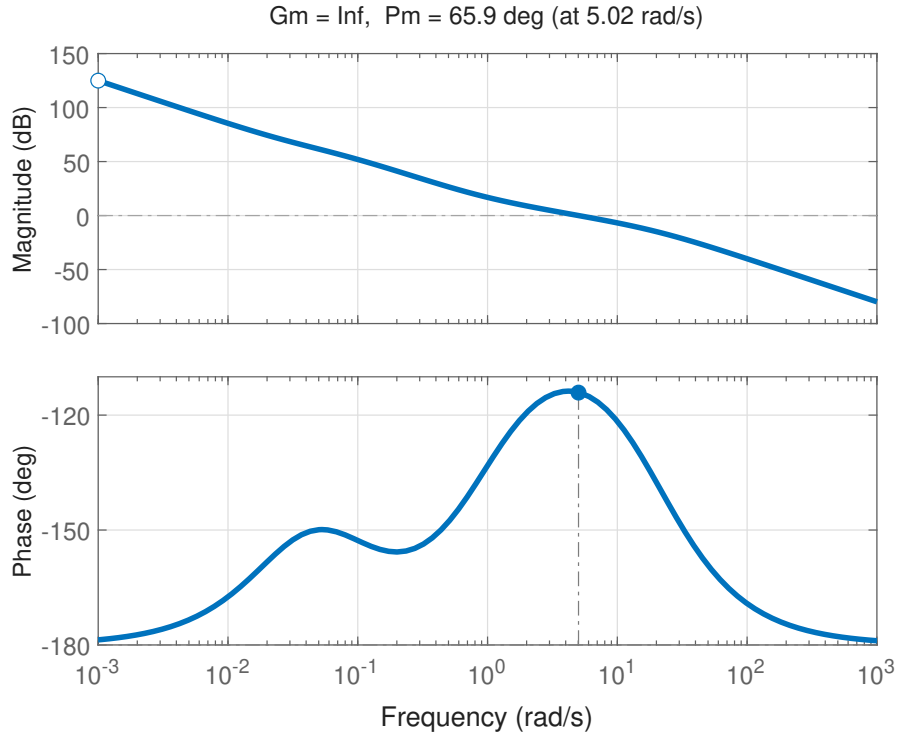


Figure 10: Bode plot of PD

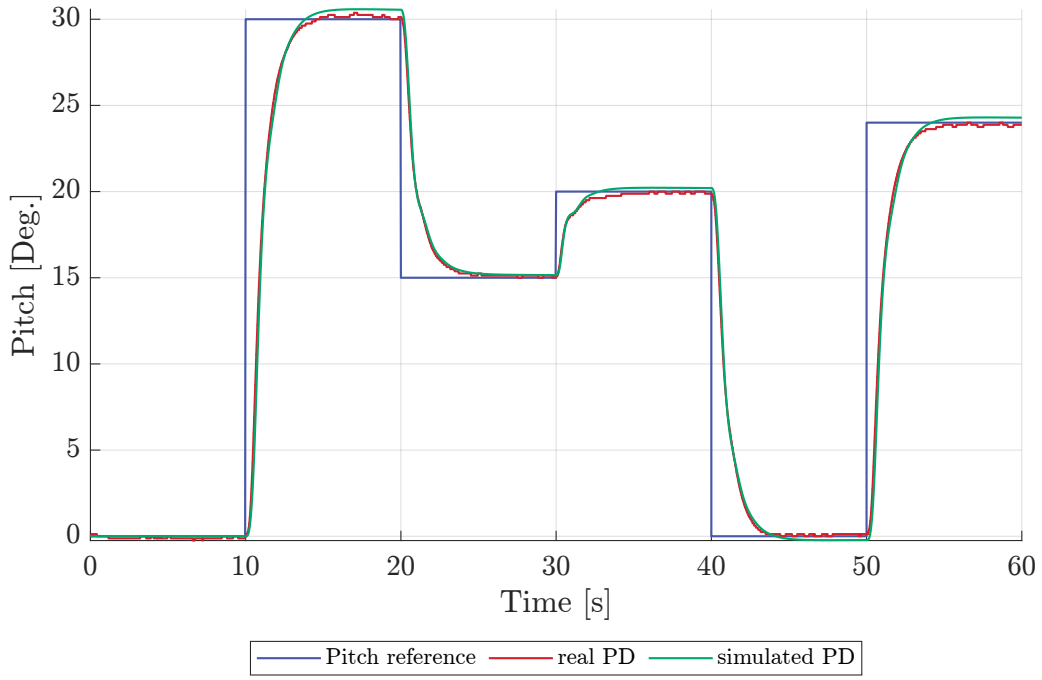


Figure 11: Step response comparison of the real and simulated system to control pitch in 1 DOF case with the PD controller

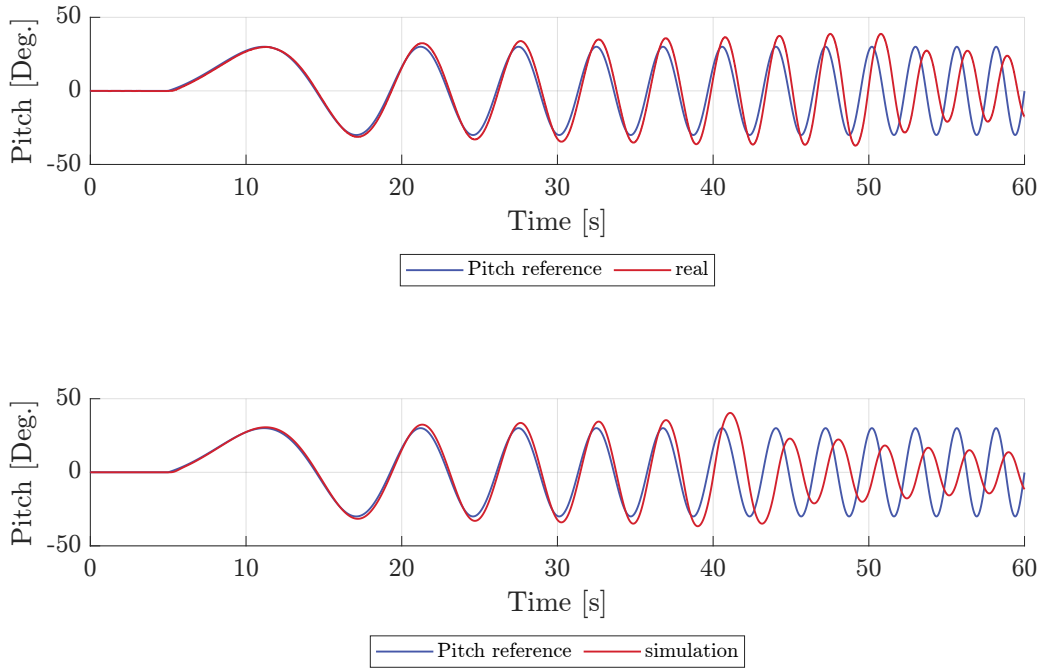


Figure 12: Comparison of the real and simulated system to control pitch in 1 DOF case with the PD controller, following a variable-frequency sine trajectory

The results are more than satisfactory.

A sensibility test was performed to understand how the different parameters of the controller change the performance of the controlled system. As expected, an increase of K_d makes for a faster response time but a noisier steady state. Increasing K_i has a similar effect in the transient but it increases the overshoot. Increasing K_d slows down the control action, making for a more cautious control but a slower response.

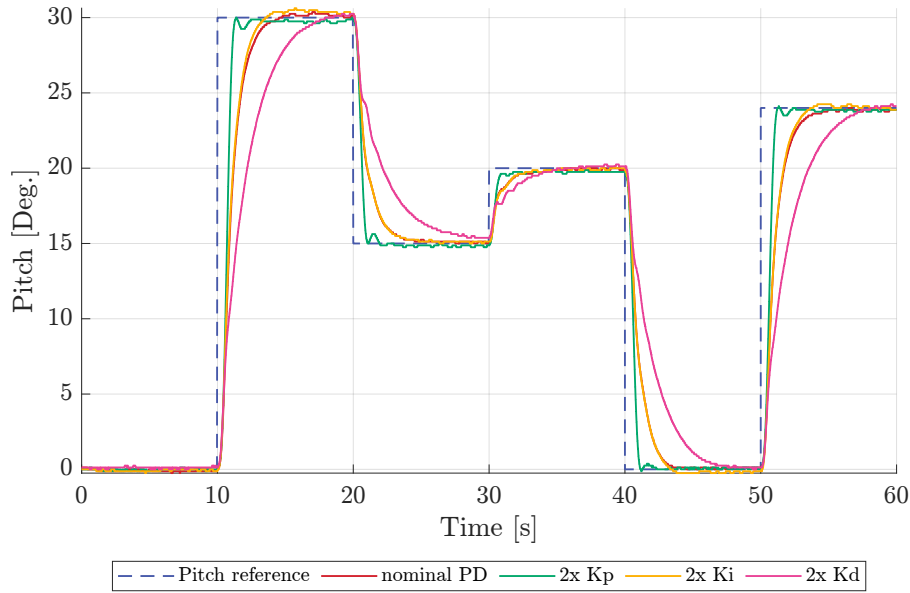


Figure 13: Comparison between variations of the selected PD in the step response

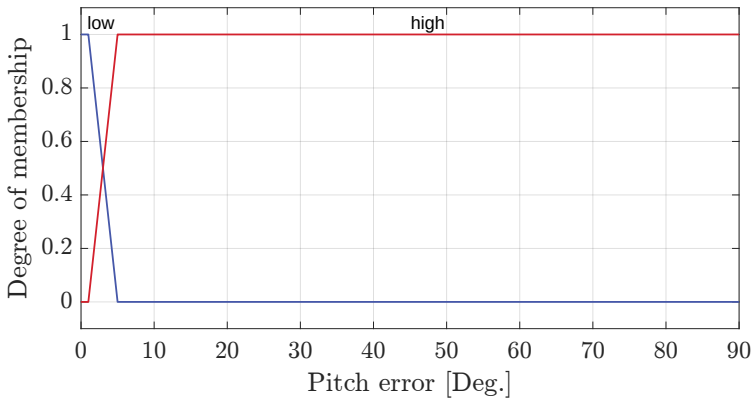
5.4. Fuzzy PID Control

The natural evolution of a PID is a fuzzy PID. The basic idea is that a specific PID controller is not generally suitable for different use cases (trajectory vs setpoint tracking), so a case varying control logic is created by changing the weights of the proportional, integral and derivative action depending on the situation the system is in. This is done using what are called "membership functions", which define some groups of the input variable. Depending on the instantaneous classification of the input variable different PID gains are obtained, interpolating for all the "grey" areas. First the membership functions were based on the error, and the feedback linearisation was kept. Then a second version was based directly on pitch position, making it possible to remove feedback linearisation.

Fuzzy PID based on error The fundamental idea behind this specific implementation is to vary the PID gains based on the absolute value of the error. In fact, in principle, a PID that behaves like a PD is more desirable during reference tracking tasks and when the absolute error is high, while if the value of the error is relatively small, a PID whose gains are more focused on steady-state convergence, with an increased integral action to act upon the mean of the error, should prove itself beneficial.

This also lends itself pretty well to be converted into fuzzy rules for the inference system, which therefore are:

- **Rule 1:** If error is *low*, then K_p is *low*, K_i is *high*, and K_d is *constant*.
- **Rule 2:** If error is *high*, then K_p is *high*, K_i is *low*, and K_d is *constant*.



Rule	K_p	K_i	K_d
1 (error low)	181886	50000	151852
2 (error high)	272830	0	151852

Table 2: Fuzzy rule-based PID gain selection

Figure 14: Membership functions related to Rule 1 and Rule 2 for the evaluation of pitch error

A significant downside of this strategy, compared to a standard PID control structure, is the increased difficulty

of tuning. This is primarily due to the higher number of parameters and their complex interdependence. This includes the definition of membership function regions, the K_d , K_i , and K_d values associated with each membership function, the number of membership functions, and the decision of whether to use only the error or also its derivative, an approach adopted in many studies (e.g. [3])

In our implementation, the derivative of the error was not included for two main reasons:

- **Numerical stability:** Derivatives are not numerically stable and are sensitive to noise and encoder quantization, which leads to spikes in error angle signals.
- **Marginal benefits:** the workaround, given that the pitch speed is an available signal is to instead use that to produce the speed error, but early simulation results showed that the inclusion of the latter term increased system complexity without providing meaningful improvements in performance.

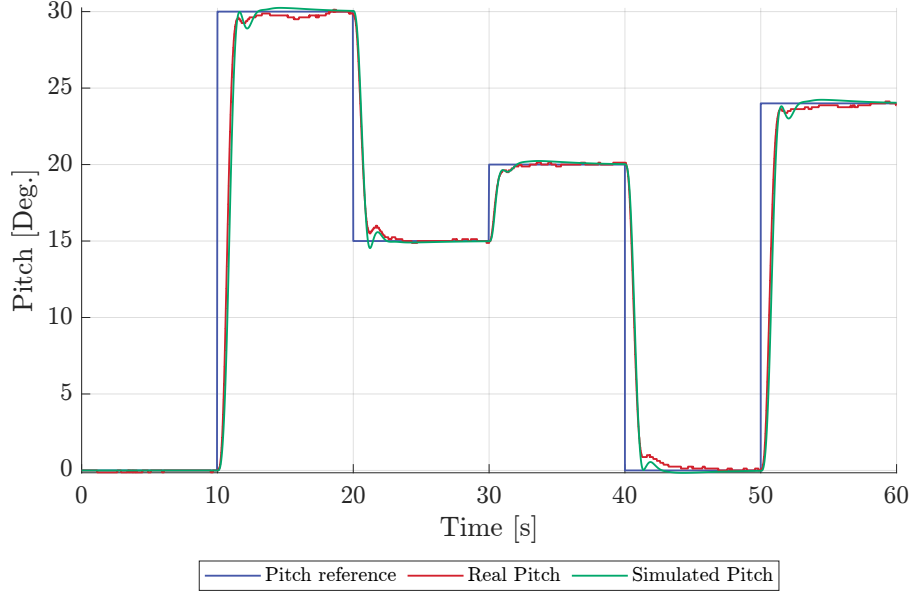


Figure 15: Fuzzy Error PID Step response

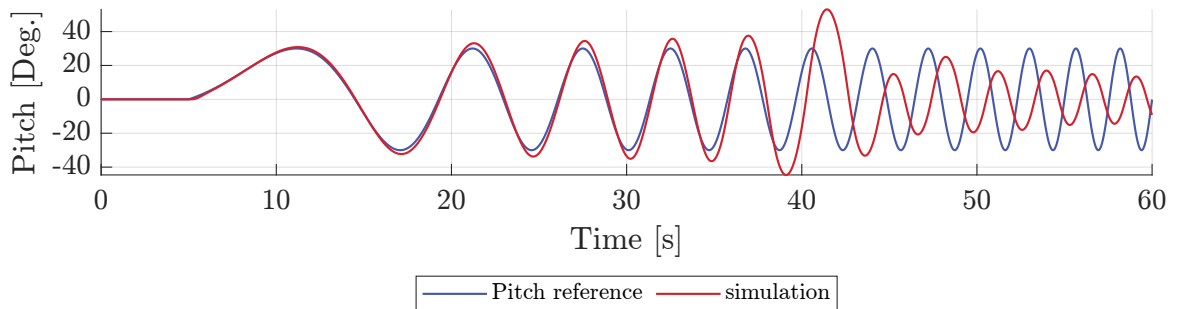
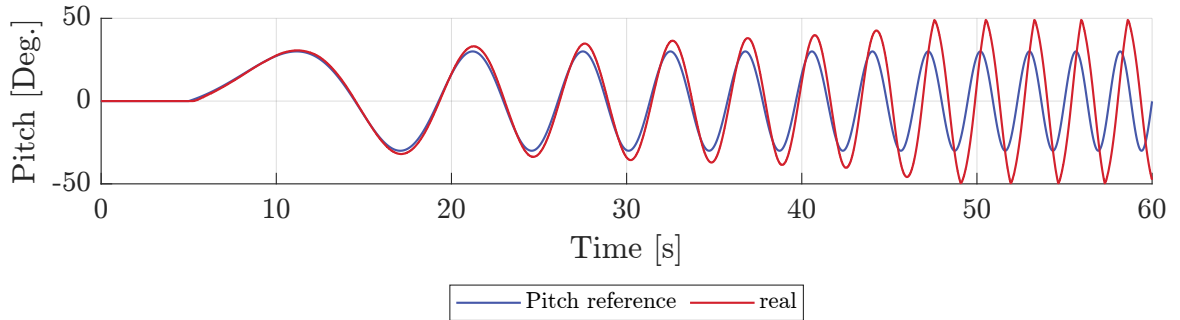


Figure 16: Fuzzy Error PID Sine response

Fuzzy PID based on position This control architecture intentionally avoids feedback linearisation. Instead, it employs a PID controller whose coefficients vary with the absolute pitch angle. A script generates an arbitrary, uniformly distributed, number of equilibrium points from which linearisations are computed. From each linearised model, PID gains are tuned using the `pdtune` function.

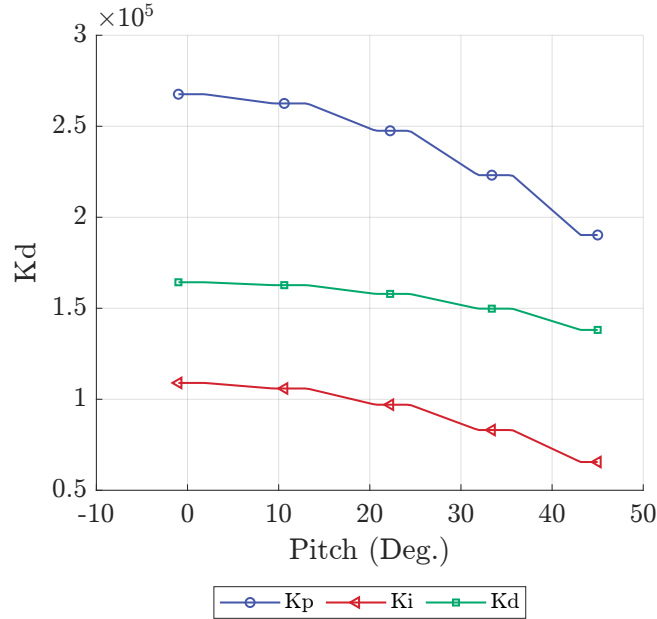


Figure 17: Position Fuzzy PID gains with respect to pitch

The resulting sets of gains are then integrated into a fuzzy inference system. This system defines equally spaced membership functions across the pitch range, with each membership function associated with a specific set of PID gains. During deployment, the absolute pitch angle is used as the inference input, leveraging the symmetry of pitch dynamics. This allows the system to only model half the pitch range (0° to 45° instead of -45° to $+45^\circ$), reducing computational effort.

In this case the main benefit for encapsulating everything inside a fuzzy inference system is having flexible interpolation of the pid gains between the reference initialization states through the membership functions.

Since everything was automated, tests in simulation were run with different amounts of initialization points and an interesting outcome is that there are not significant differences between 200 and 4 initialization points in terms of the behaviour of the resulting controller.

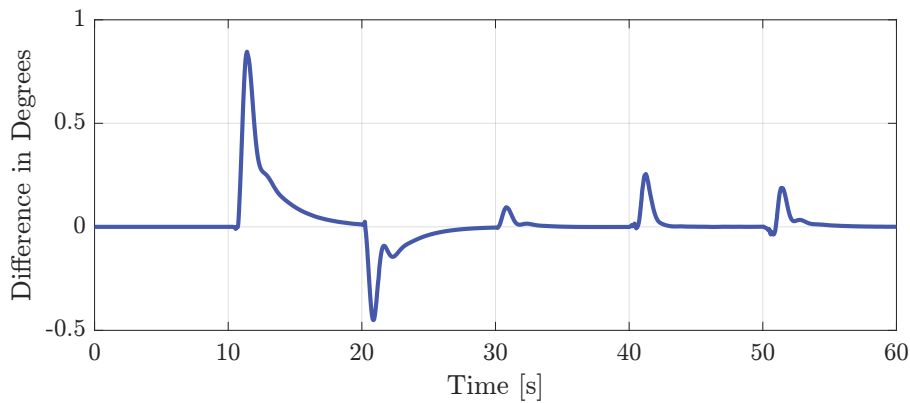


Figure 18: Difference between simulated Pitch obtained with 200 linearisations vs. 4 linearisations. It is minimal. Maximum step of the reference is of 30° at 10 seconds

Experimentally the same conclusion was essentially reached:

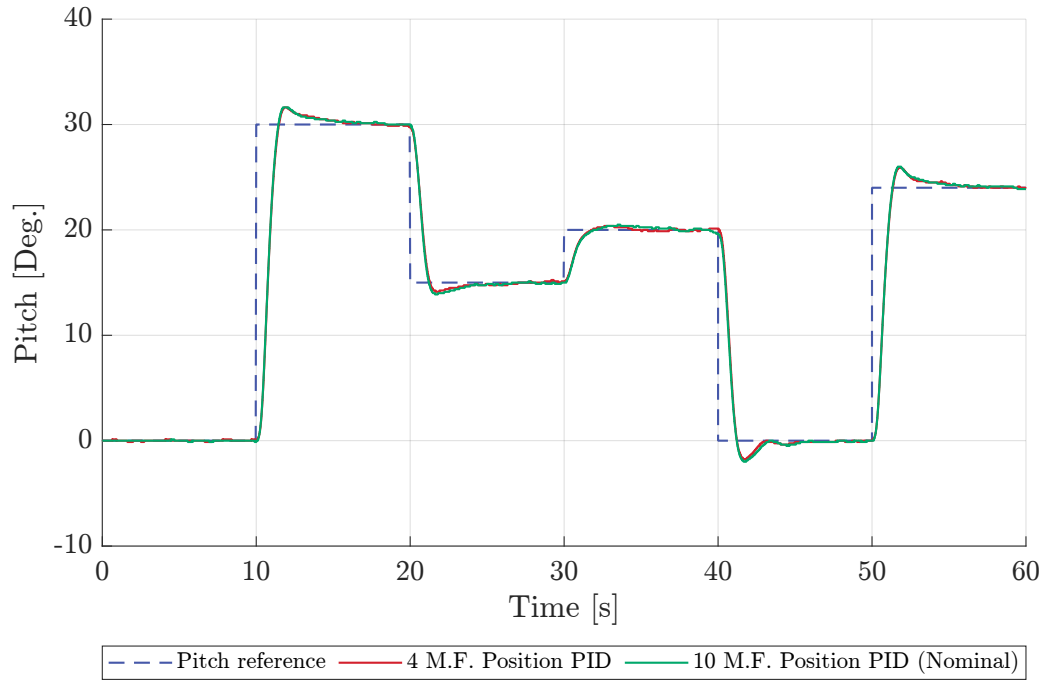


Figure 19: Comparison between 4 linearisation positions and 10 linearisation positions

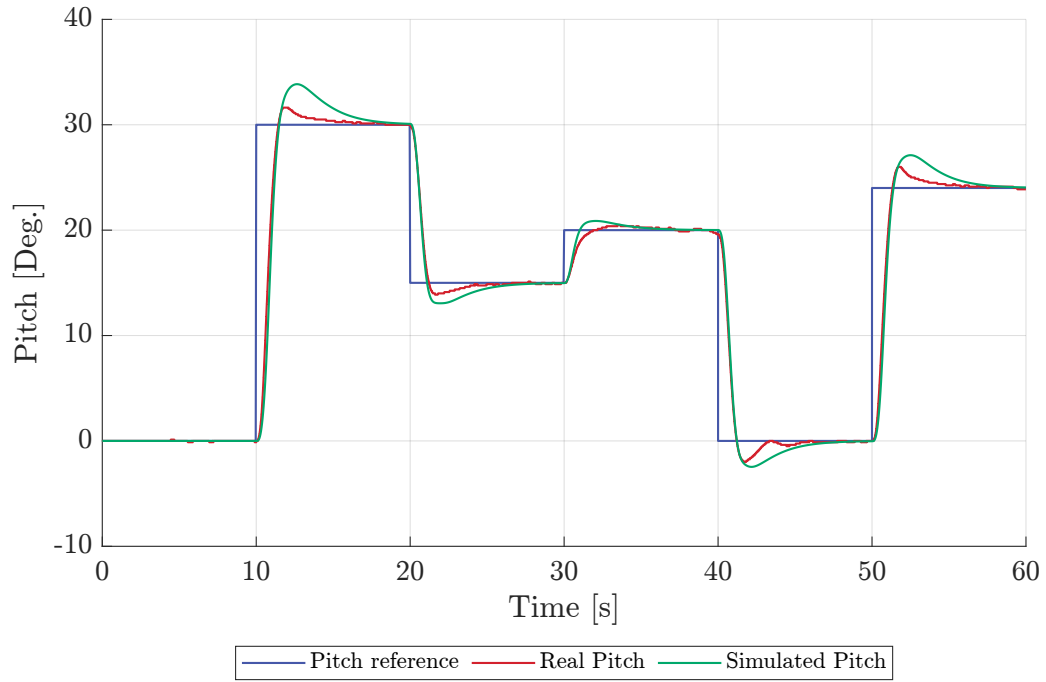


Figure 20: Comparison between Pitch 1 DOF controlled through Fuzzy Position PID in the real and simulated system for a Step response

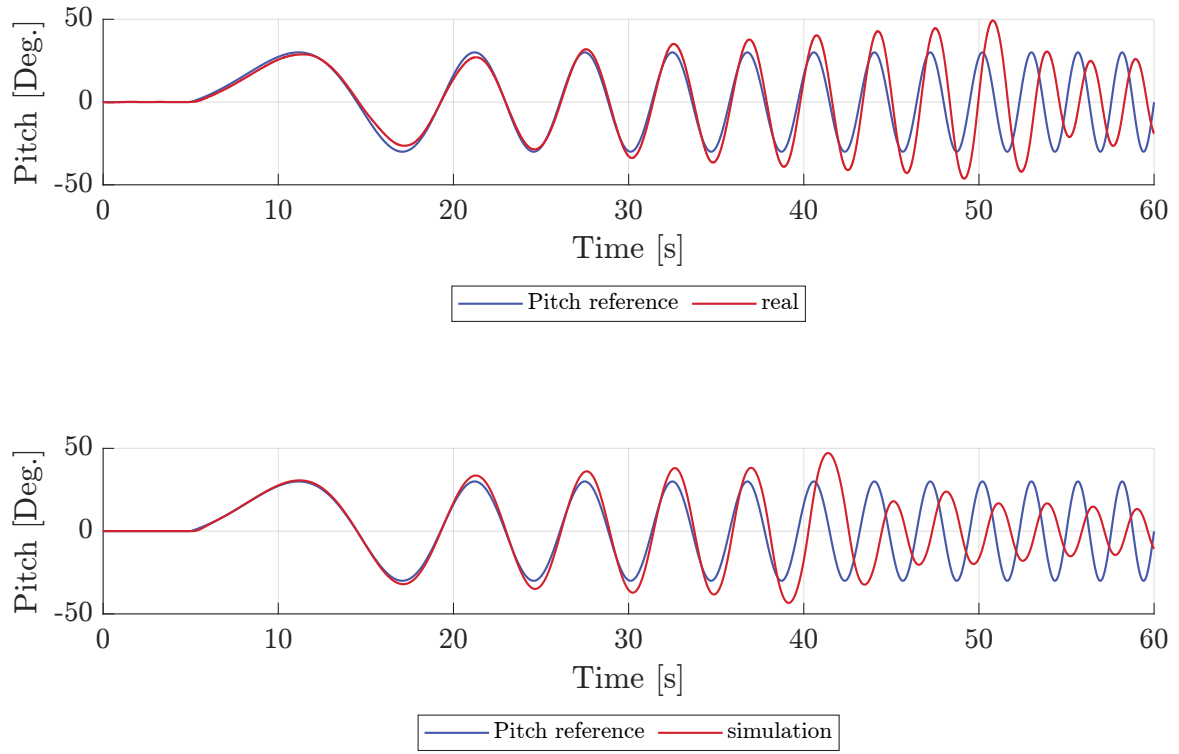


Figure 21: Comparison between Pitch 1 DOF controlled through Fuzzy Position PID in the real and simulated system for a Sine response

5.5. Model Predictive Control (MPC)

MPC is a control strategy that employs an optimisation problem to determine the next control action. The basic idea is to have a cost function that determines what "good" control means and then MPC simulates the model forward, finding the (locally) optimum sequence of inputs. The optimisation problem that was set up was:

$$\begin{aligned}
 & \min_{\{V_k\}_{k=0}^{N-1}} \sum_{k=0}^{N-1} (w1 * e_k^2 + w2 * \dot{\theta}_k^2 + w3 * \lambda I_{e,k}^2) & (16) \\
 \text{subject to:} \quad & -18 \leq \{V_k\}_{k=0}^{N-1} \leq 18 & \text{(voltage saturation)} \\
 & \theta_{k+1} = f(\theta_k, \dot{\theta}_k, \Omega_{mk}, V_k) & \text{(system model)} \\
 & I_{e,k+1} = I_{e,k} + e_k & \text{(integrator of the error)} \\
 & e_k = \theta_{\text{ref},k} - \theta_k & \text{(tracking error)}
 \end{aligned}$$

The MPC has a time horizon window of 1 second, divided into 10 time intervals of 0.1 seconds. Adding an integral error improved significantly the steady state tracking, and adding the pitch speed to the cost function improved the overshoot, but unfortunately it was not enough to improve the performances, highlighting some unmodeled dynamics in the real system, since oscillations were inevitable. A very long compilation time also made testing very time consuming, but very little seemed to change between different setups.

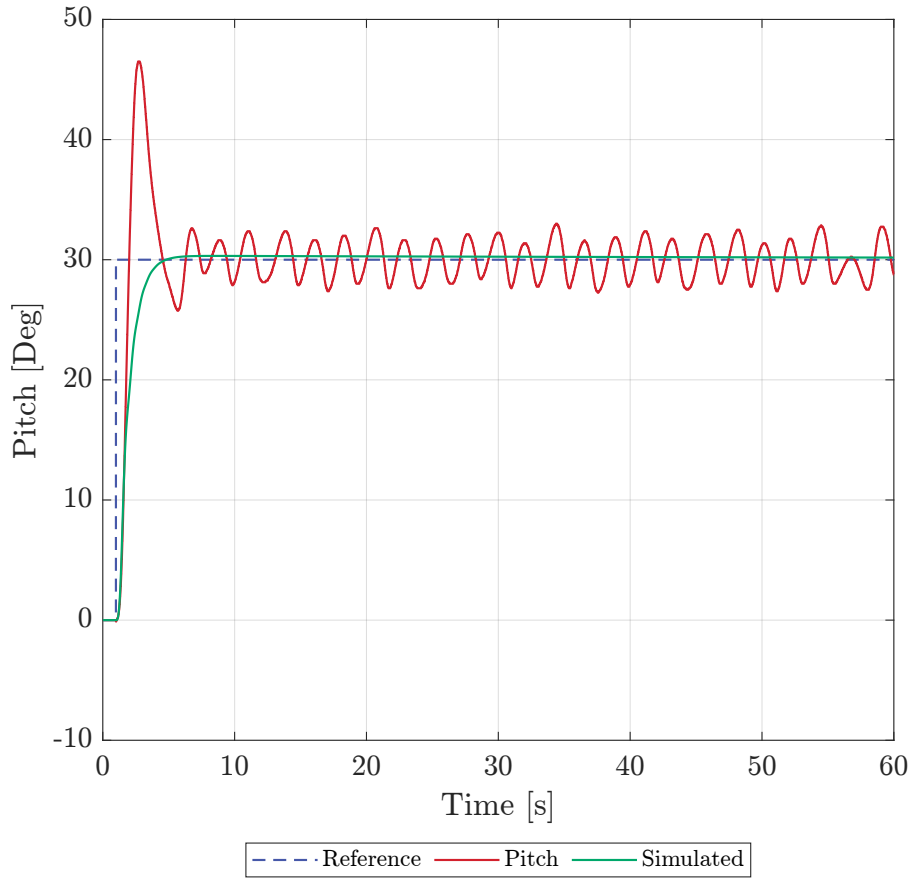


Figure 22: Pitch controlled through MPC, with a 4 degrees band

What makes it even clearer that this is a model limitation is the fact that in the simulation the performance is significantly better, since the model MPC performs the optimisation on corresponds exactly to the one it controls.

6. 2 DOF Helicopter Control

In this section the strategies for controlling the system with both DOF unlocked will be outlined. First, it is necessary to perform parameter identification for the new parameters involved when the support yoke is allowed to rotate. Then, different families of controllers are tested and implemented.

With both degrees of freedom unlocked, the system becomes a MIMO system which adds extra challenges to the control problem. The first control strategy introduced builds two independent cascaded PID controllers, decoupled through feedforward terms. Then MIMO centralized control strategies were implemented: starting with LQR, then using LMIs to compute feedback gains, LQG for cases where the required states aren't all measurable, time-varying LQ (LQ-TV) for trajectory tracking, and finally, quasi-infinite horizon non-linear MPC (QIH-NMPC). A brief summary of the basic ideas behind each controller will be given, and then the results will be shown both in simulation and on the real system.

Since the dynamics of the propellers are much faster than the rest of the system, two different control paradigms can be modeled and implemented:

- building an inner loop for the propellers' speed and an outer loop for the rest of the system
- integrating the dynamics of the propellers directly in the mechanical model of the system and implementing only one feedback loop

Starting from the mathematical formulation of the Quanser Aero described in detail in [Sec. 2], a Simulink model was built for simulation and control of the system following each paradigm.

Cascaded Control Loop Strategy The fast dynamics of the propellers allow a cascade of control loops to be built: the inner loop, which provides the input voltage for the propeller, built as explained in [Sec. 3.1]; and the outer loop, which provides the reference motor speeds to the control of the rest of the system.

Single Control Loop Strategy In this scenario the dynamics of the propeller are integrated in the system model so that there is a single control loop which provides the voltages.

6.1. Parameter Identification

6.1.1. Yaw Model Parameters

The first thing to estimate is the behavior of the yaw, and all the parameters connected to it. The important parameters to gather for the 1 DOF yaw model are the damping coefficient and the friction. To derive them we have locked the pitch motion, followed the same procedure we followed to derive the 1DOF pitch model and found:

$$\ddot{\psi}(J_{z_1} + J_{z_2}) = F_v L_m - \dot{\psi} \beta_\psi - \text{sign}(\dot{\psi}) \mu \quad (17)$$

What needs to be estimated is the damping factor β_ψ , and a model for the friction μ . To derive them a dataset of steps was gathered, letting the yoke reach the terminal velocity before increasing to the next step. The best way to model the friction μ was to use static and dynamic friction, assuming that the friction to start the movement was higher, and then it suddenly decreased:

$$\mu(x) = \begin{cases} \dot{\psi} * \beta_\psi - F_v * L_m, & \text{if } \sum_{i \neq s} \text{moment}_i < \mu_s \\ \mu_d, & \text{otherwise} \end{cases}$$

To find the static friction the force applied when the system first started moving was computed, then for the dynamic coefficients an optimization problem to find the (locally) optimum pair of values to match the simulation and the real data was formulated:

$$\begin{aligned} & \min_{\beta_\psi, \mu_d} \sum_{i=0}^{i=N} [(\psi_{\text{simulated}}^i - \psi_{\text{real}}^i)^2 + (\dot{\psi}_{\text{simulated}}^i - \dot{\psi}_{\text{real}}^i)^2] \\ & \text{subject to } \psi_{\text{sim}}^i = \text{model}(\psi^{i-1}; \beta_\psi, \mu_d), \\ & \quad \dot{\psi}_{\text{sim}}^i = \text{model}(\dot{\psi}^{i-1}; \beta_\psi, \mu_d), \\ & \quad \beta_\psi > 0, \\ & \quad \mu_d > 0 \end{aligned}$$

The results of the optimization problem greatly improved when we separated the dataset into positive and negative $\dot{\psi}$ directions, we therefore assumed that the friction had to be different in the two directions. Since this is a very high nonlinearity in control we used an average of the two, and only used the full model for the simulation.

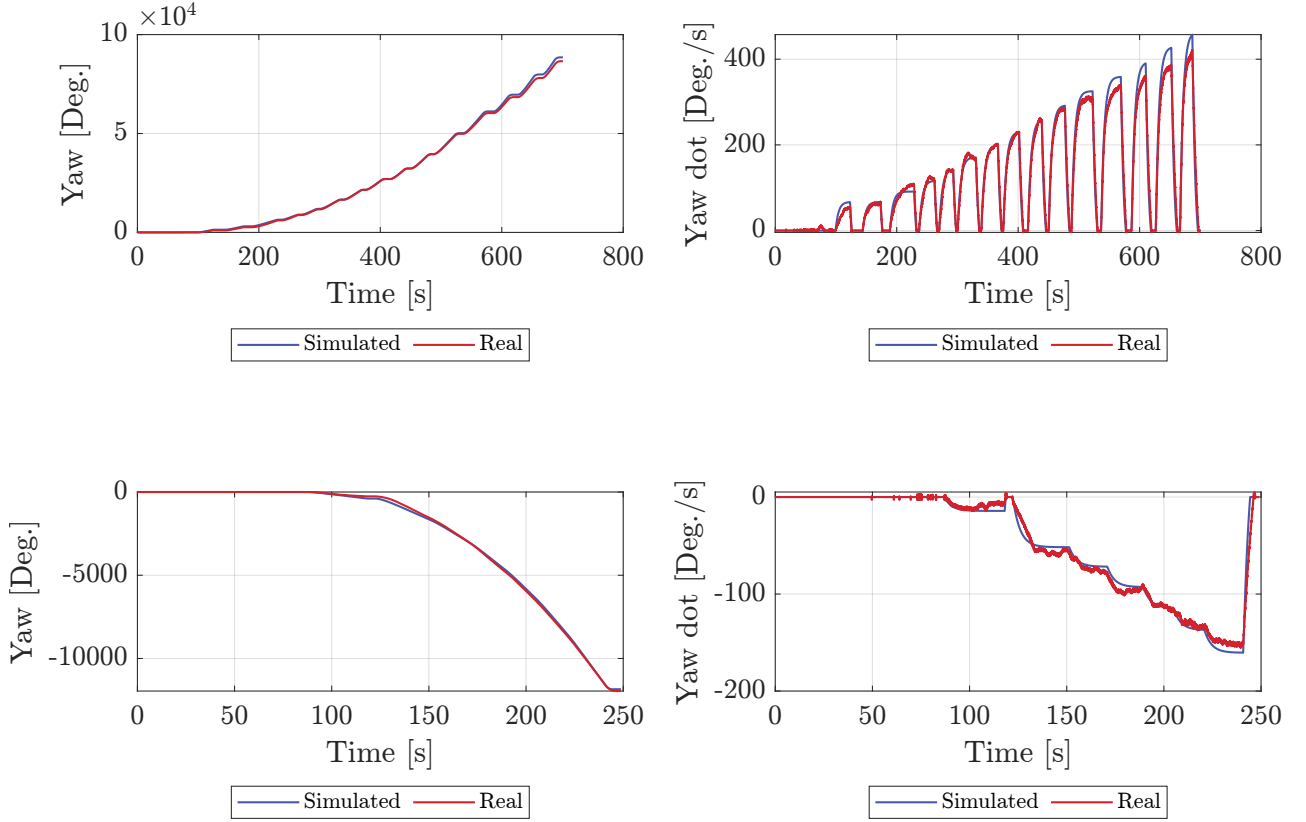


Figure 23: Comparison between the final simulation and the real data on the positive and negative voltage dataset

6.1.2. Coupling Torque Coefficients

The other important parameters that are only relevant in 2DOF are the coupling torques, which are the most challenging to estimate. The model of the torque is:

$$Q_i = \gamma_i * \Omega_m^2 \quad (18)$$

Note that for this model we have included the possibility that $\gamma_v \neq \gamma_h$ to improve the model further. This was not necessary for the force as the overall results were not as sensitive to a small difference between the fans. At first we have estimated γ_v by locking the yaw and running the vertical fan, then we estimated γ_h from a 2DOF dataset, where both pitch and yaw were free to move. To do so accurately we imposed that at every step the simulated pitch was equal to the real pitch, so that there would be no propagation of errors. The reason why the yaw dataset is computed with also the pitch free to move is that the coupling is not enough to beat the friction at most voltages, therefore the data points we could gather were limited.

From the first we estimated γ_h , from the second we estimated γ_v , and then we used the third to validate. In both cases we set up an optimization problem:

$$\begin{aligned} \min_{\gamma_i} \quad & \sum_{i=1}^n (y_i^{\text{sim}} - y_i^{\text{real}})^2 \\ \text{subject to} \quad & y_i^{\text{sim}} = f(x_i; \gamma_i) \\ & \gamma_i \in \Gamma \end{aligned}$$

Where $y_i = \psi$ for the first dataset and $y_i = \theta$ for the second dataset. As we can see from the figures below the results are pretty accurate when it comes to fixing one degree of freedom and simulating the other, but it is very hard to simulate both degree of freedom fully, as yaw speed is influenced by pitch position, and pitch speed is influenced by yaw speed, so any small error easily propagates.

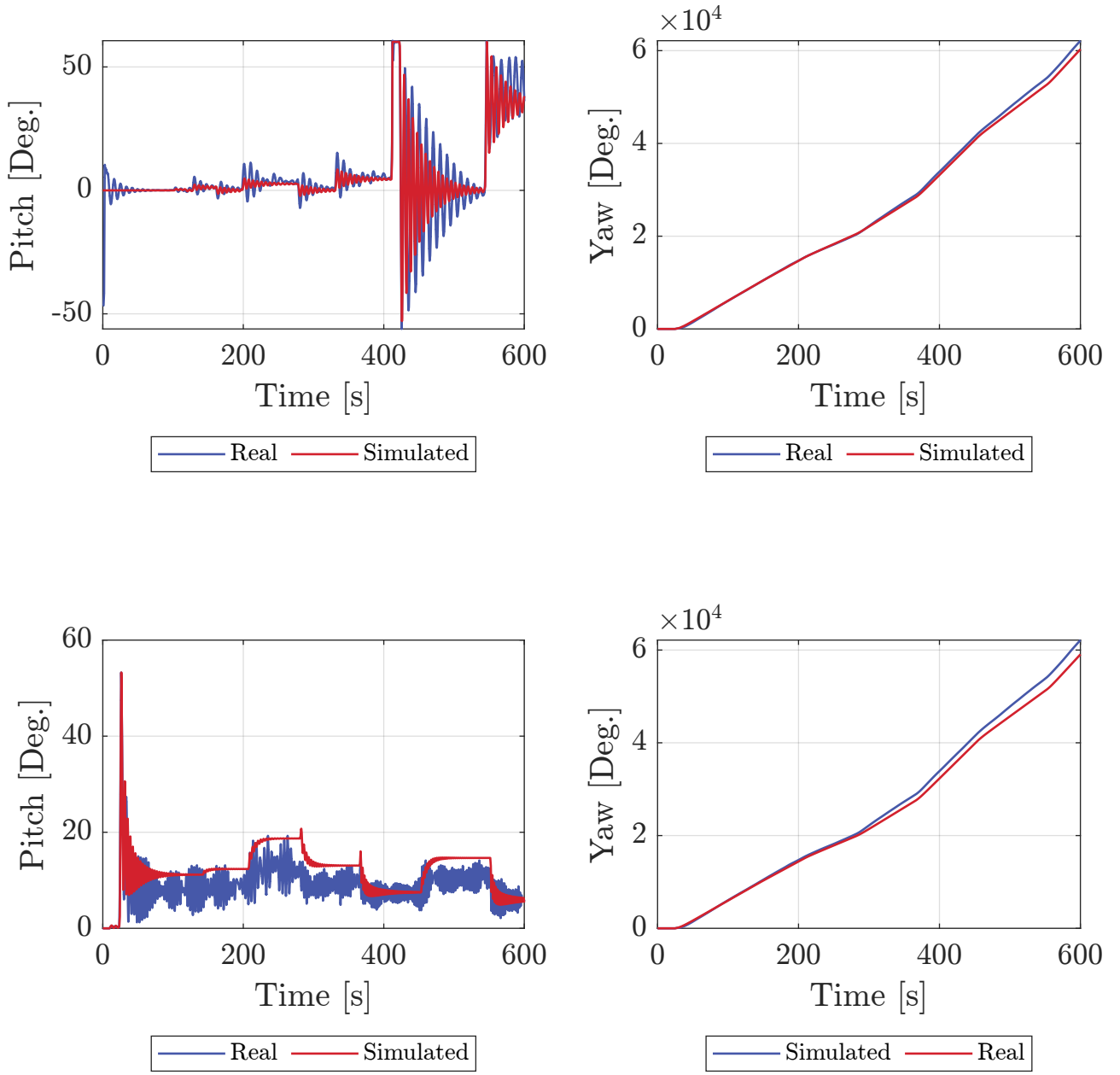


Figure 24: Comparison between the simulation and the real data when one degree of freedom is fixed(top) and fully simulated, letting the degrees of freedom influence each other (bottom)

6.2. PID Control with Decouplers

The basic idea behind this strategy of control is to control the linear part of each degree of freedom through PID control, and compensate any coupling and nonlinearity in feedforward.

The feedforward has been computed from the following formulas (a simplified version of 6 and 7):

$$\begin{aligned} \ddot{\psi} * J_{eqy1} + \dot{\psi} \dot{\theta} \sin 2\theta * J_{eqy2} + \dot{\psi} \beta_{\psi} &= F_v \cos \theta L_m - Q_h \cos \theta - \mu \\ \ddot{\theta} * J_{eqp1} - \frac{1}{2} \dot{\psi}^2 \sin 2\theta * J_{eqp2} + \dot{\theta} \beta_{\theta} + M_{body} g D_m \sin \theta &= F_h L_m - Q_v \end{aligned} \quad (19)$$

We note that J_{eqy1} is a function of pitch, but we will ignore this consideration for the scope of this controller because it would render the tuning of the PID impossible, a clear limitation of this method. From these equations we see that the compensation of the nonlinearities of the pitch motion comes down to

$$Pitch_{comp} = M_{body} g D_m - \frac{1}{2} \dot{\psi}^2 \sin 2\theta * J_{eqp2} + Q_v \quad (20)$$

The yaw motion is slightly more delicate because the arm of the moment caused by the fan changes with the pitch, so both the coupling force F_h and the coupling torque Q_h are multiplied by $\cos(\theta)$. Taking this into consideration the compensation comes down to:

$$\psi_{comptot} = \frac{\mu_s + \dot{\psi} \sin 2\theta * J_{eqy2}}{\cos(\theta)} + Q_v \quad (21)$$

Since $\theta \in [-45^\circ, 45^\circ]$ the division by $\cos(\theta)$ does not cause singularities.

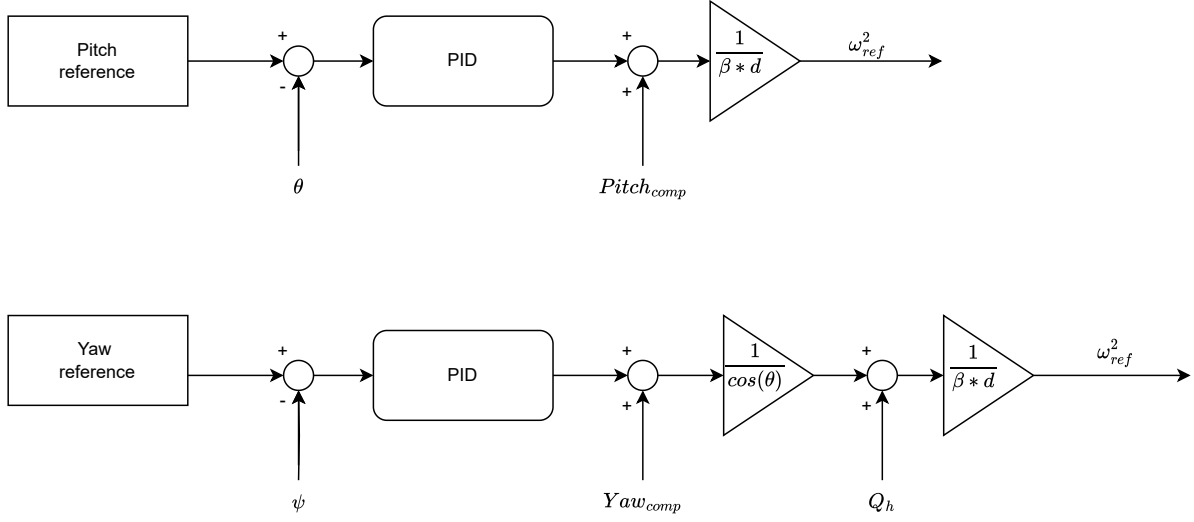


Figure 25: Compensation control scheme

Other than the big approximation of a constant yaw inertia another big limitation of this system was the fan speed control, since the reference it had to follow was highly non linear, as compensations were added the omega reference got harder and harder to follow accurately, and the overall system became not robust. For a first attempt the 1DOF PD were used to control both degrees of freedom, with the intention of later tuning more specific PIDs, but then it became clear that another approach was necessary to maintain an acceptable control speed.

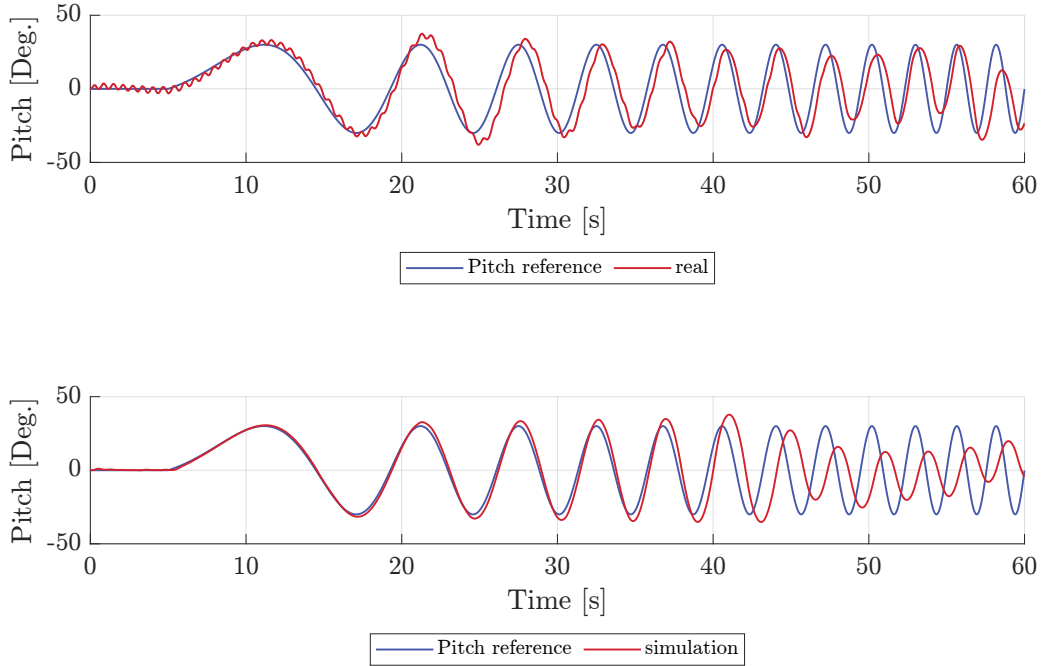


Figure 26: Comparison between Pitch 2 DOF controlled through PID in the real and simulated system for a sine response

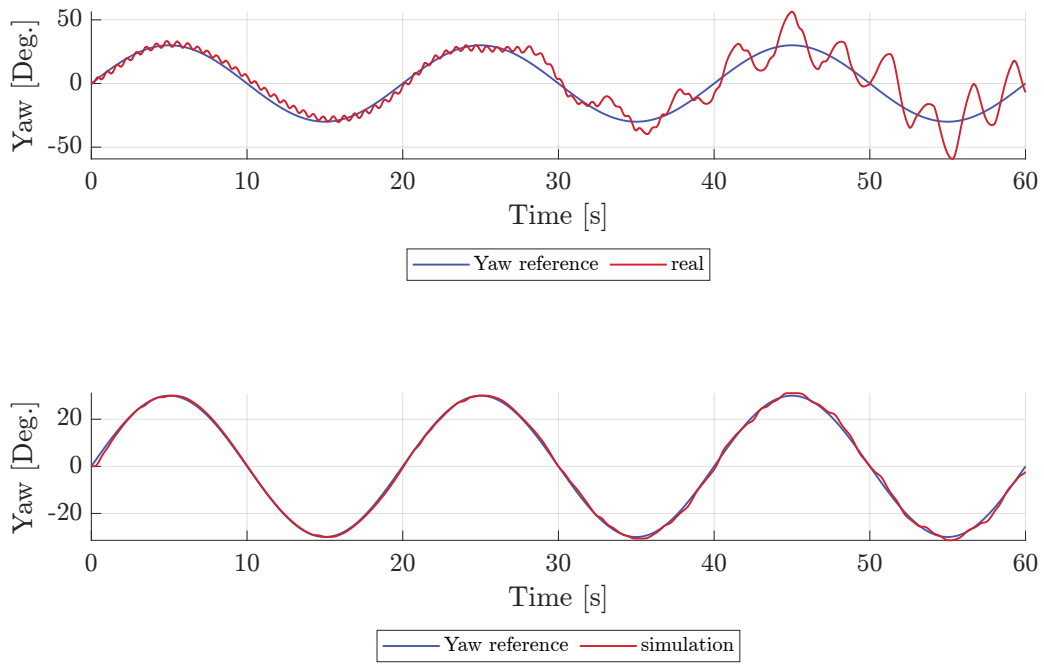


Figure 27: Comparison between Yaw 2 DOF controlled through PID in the real and simulated system for a sine response

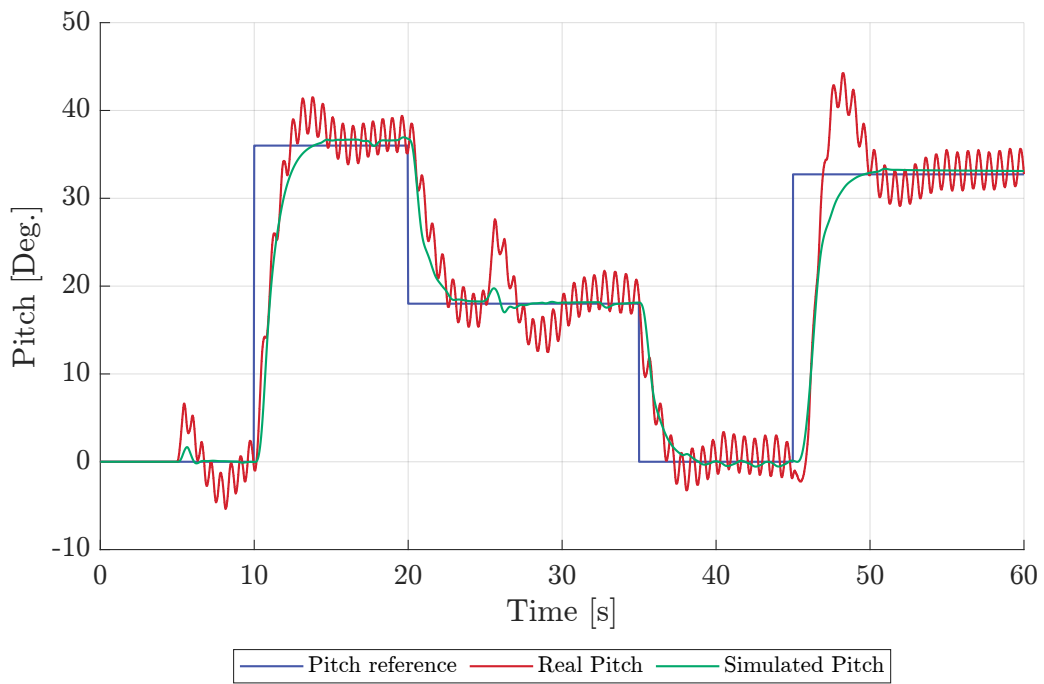


Figure 28: Frequency response

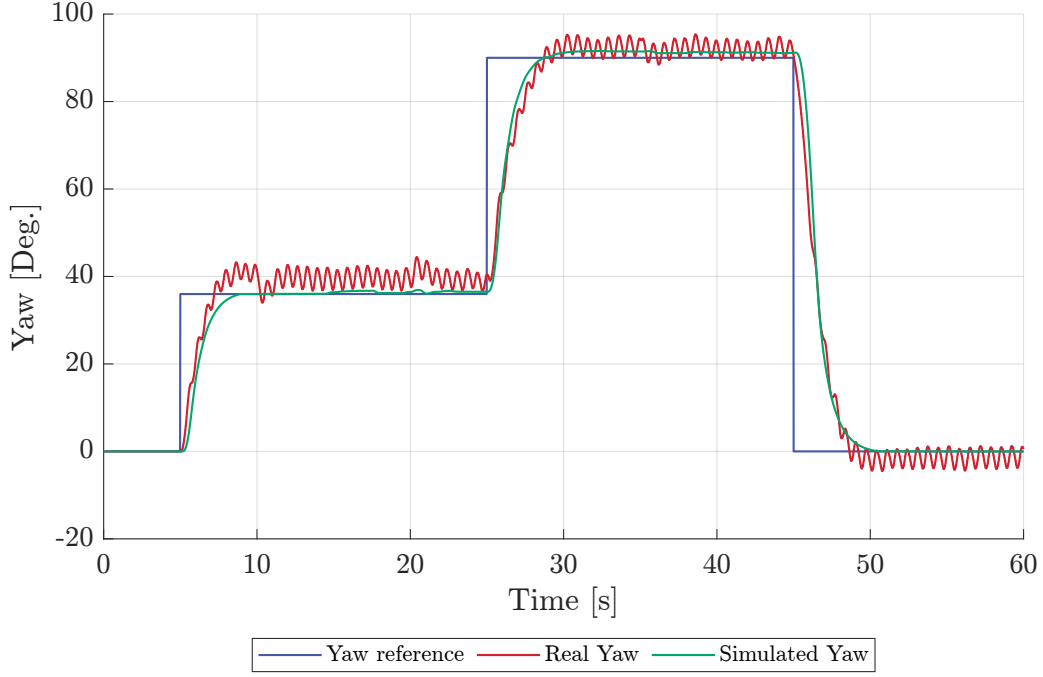


Figure 29: Step responses

6.3. Control with LMIs

Using linear matrix inequalities, it is possible to specify a region in the S-plane where we want the poles of the system to be placed in order to fulfill some desired specifications. That is, the desired settling time, T_{sett} , and damping can be satisfied using the linearised system matrices and the two parameters, α and ϕ . α is the magnitude of the maximum negative spectral abscissa and ϕ defines the slope of a cone-shaped region of D-stability as shown in Figure 30 below.

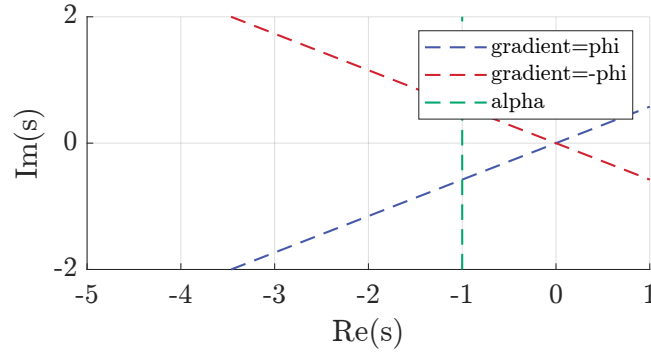


Figure 30: Boundaries of region specified by LMIs

The formulation of LMIs is shown below, where a higher α corresponds to a more stable closed-loop system, and a lower ϕ corresponds to a smaller amplitude of oscillations.

$$\alpha = \frac{5}{T_{\text{sett}}} \quad (22)$$

$$QA^T + L^T B^T + AQ + BL + 2\alpha Q < 0 \quad (23)$$

$$\begin{bmatrix} \sin(\phi)((AQ + BL) + (AQ + BL)^T) & \cos(\phi)((AQ + BL) - (AQ + BL)^T) \\ -\cos(\phi)((AQ + BL) - (AQ + BL)^T) & \sin(\phi)((AQ + BL) + (AQ + BL)^T) \end{bmatrix} \prec 0 \quad (24)$$

where the decision variables are Q , a positive definite matrix, and L , the dot-product of Q and the desired gain K such that

$$K = LQ^{-1} \quad (25)$$

A minimization problem can also be added to minimize the control effort while remaining in the specified region. Since minimizing the norm of the gain cannot be directly formulated as an LMI, a weighted sum of the upper bounds, k_L and k_Q , of the norms of the decision variables is minimized instead.

$$\begin{aligned} \min_{k_L, k_Q} \quad & \gamma_L k_L + \gamma_Q k_Q \\ \text{with } & \gamma_L = 0.1 \text{ and } \gamma_Q = 10 \end{aligned} \quad (26)$$

To upper bound the norms of L and Q by $\sqrt{k_L}$ and k_Q respectively, the following LMIs are added to the problem.

$$\begin{bmatrix} k_L I & L^T \\ L & I \end{bmatrix} \succeq 0 \quad (27)$$

$$\begin{bmatrix} k_Q I & I \\ I & Q \end{bmatrix} \succeq 0 \quad (28)$$

The solution to the problem is computed using the Yalmip solver add-on in MATLAB. The initial idea was to use the linearised model introduced in Section 4.3 evaluated at the equilibrium point for which $\theta = 0.2$, $\psi = 0$, and the system is still, with the feedback control law $\mathbf{u} = K \mathbf{x}$ such that $\dot{\mathbf{x}} = (A + BK)\mathbf{u}$, where $\mathbf{x} = [\theta, \dot{\theta}, \psi, \dot{\psi}, \Omega_0, \Omega_1]^T$ and $\mathbf{u} = [V_0, V_1]^T$.

Gains computed with different desired settling times and damping angles were obtained and applied to the system in simulation. Since the model is linearised around a pitch angle of 0.2 radians and a yaw angle of 0 radians, the system is fed a reference trajectory that moves the The results were then inspected to select the gains to be applied on the real system for an adequate analysis.

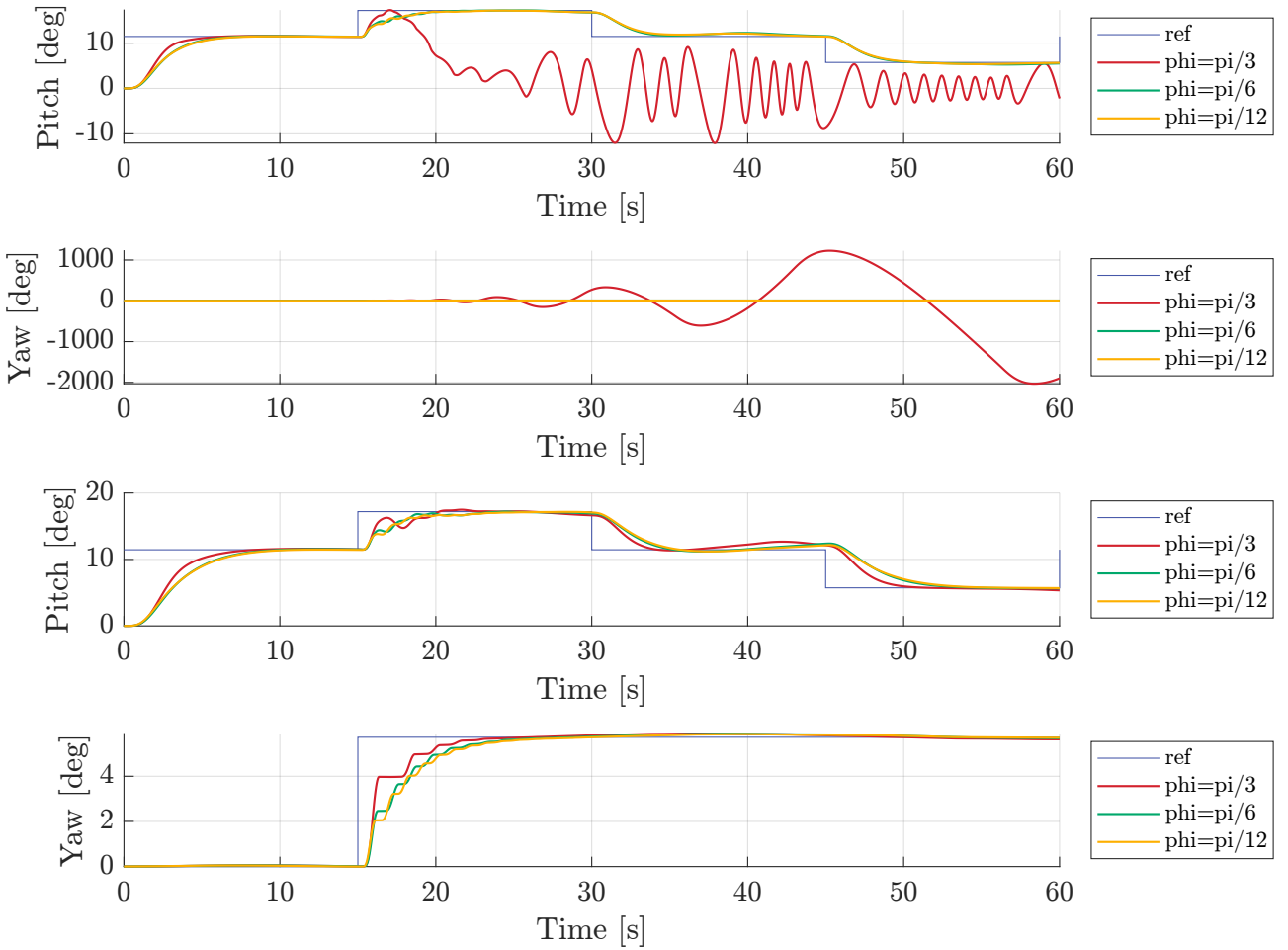


Figure 31: Simulation responses to the same reference with gains computed for different damping angles and settling times. First pitch-yaw pair for $T_{\text{sett}} = 7$ and second pair for $T_{\text{sett}} = 10$

The simulation results show an unstable yaw behavior and, accordingly, unexpected pitch behaviour when the settling time is set to 7 and the damping angle is set to $\pi/3$. This can be attributed to the equilibrium point

around which the system was linearized being only locally asymptotically stable, indicating that the change in the reference signal took the system outside the region of attraction in these cases. Specifically, the values of $\dot{\theta}$ and $\dot{\psi}$ move too far away from zero. Due to these findings, the real gains computed for $\phi = \pi/3$ are not tested on the real system; the rest are applied, and the results are shown below.

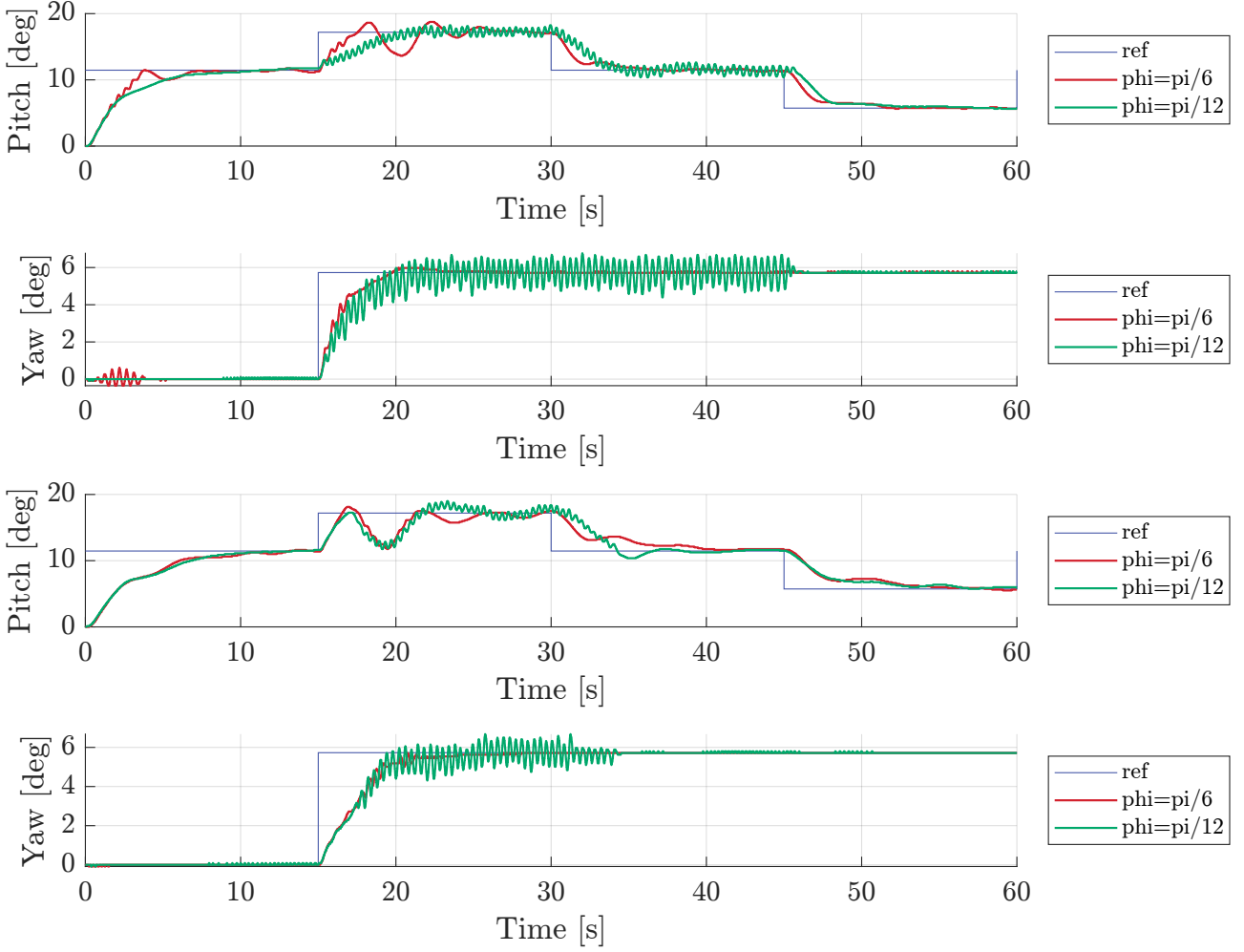


Figure 32: Real system responses to the same reference with gains computed for different damping angles and settling times. First pitch-yaw pair for $T_{\text{sett}} = 7$ and second pair for $T_{\text{sett}} = 10$

The tradeoff between the system's ability to quickly reach the setpoint and the frequency at which it oscillates around it is more apparent compared to the simulation. The results of this control scheme showed an overall unsatisfactory performance and lots of oscillations. Therefore, an alternate control scheme using cascaded control loops was proposed. The speeds of the motors are controlled by an inner PID controller while the outer loop uses gains computed with LMIs to provide a reference motor speed rather than a reference voltage. Therefore, the system now has 6 states instead of 8, as voltage calculation is dealt with by the faster inner loop with a cutting frequency of 50 rad/s, and the motor dynamics do not have to be accounted for or modeled. The new linearized system is that presented in Section 4.3.1, again evaluated at the equilibrium point for which $\theta = 0.2$, $\psi = 0$, and the system is still, with the feedback control law $\mathbf{u} = K\mathbf{x}$ such that $\dot{\mathbf{x}} = (A + BK)\mathbf{u}$, where $\mathbf{x} = [\theta, \dot{\theta}, \psi, \dot{\psi}]^\top$ and $\mathbf{u} = [\Omega_0^2, \Omega_1^2]^\top$, and the gains are calculated using the same LMIs. The results in simulation are plotted below.

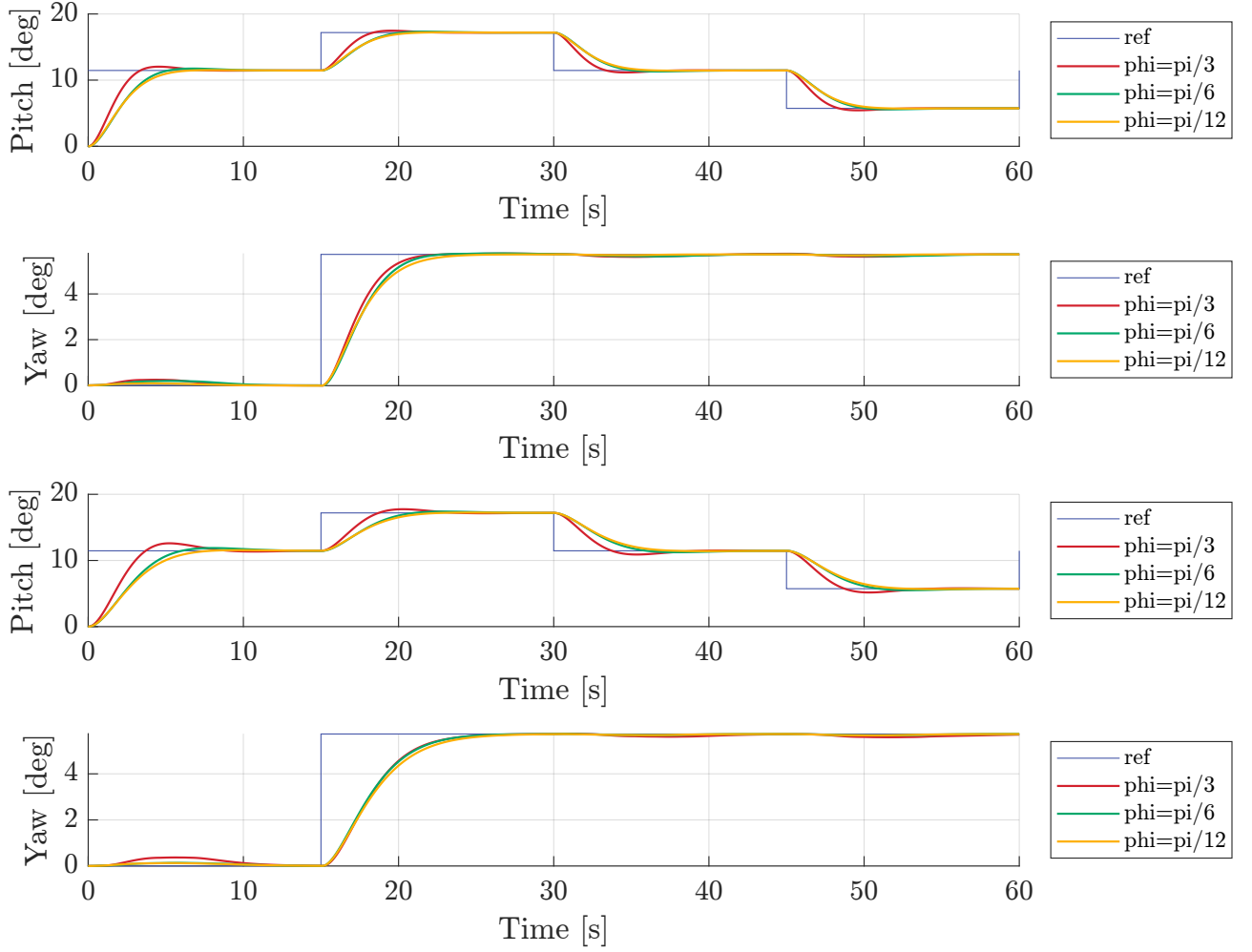


Figure 33: Simulation responses to the same reference with gains computed for different damping angles and settling times to find the motor speed references. First pitch-yaw pair for $T_{\text{sett}} = 7$ and second pair for $T_{\text{sett}} = 10$

This setup does not present any issues with the simulation (in terms of instability) so the controllers are all applied to the real system and the response is shown below. The response using the gain computed for a settling time of 7 seconds and damping angle equal to $\pi/6$ seems to provide the best results. In general, the system performs much smoother than in the previous configuration.

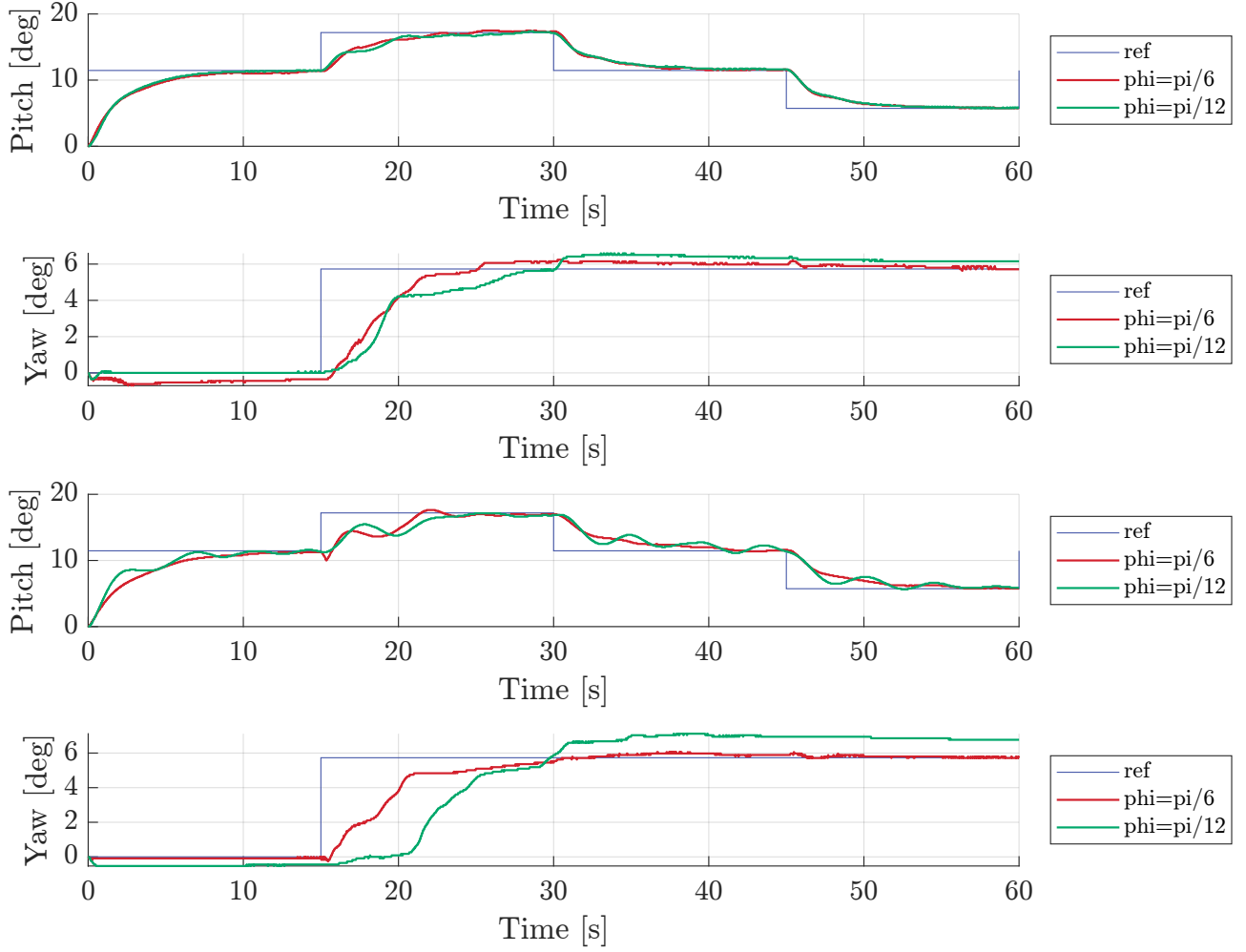


Figure 34: Real system responses to the same reference with gains computed for different damping angles and settling times to find the motor speed references. First pitch-yaw pair for $T_{\text{sett}} = 7$ and second pair for $T_{\text{sett}} = 10$

6.4. LQ Control

The key idea behind Linear Quadratic (LQ) control is to design an optimal static state-feedback control law for a linear system of the form:

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u},$$

where $\mathbf{x} \in \mathbb{R}^n$ is the state vector and $\mathbf{u} \in \mathbb{R}^m$ is the control input.

The LQ controller is obtained by solving a minimization problem over a quadratic cost function:

$$\min_{\mathbf{u}} J = \int_0^\infty (\mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{u}^\top \mathbf{R} \mathbf{u}) dt,$$

where $\mathbf{Q} \succeq 0$ and $\mathbf{R} \succ 0$ are symmetric weighting matrices that penalize the state deviation and control effort, respectively.

The optimal control law has the form:

$$\mathbf{u} = -\mathbf{K}\mathbf{x},$$

where $\mathbf{K}$ is the **feedback gain matrix** computed by solving the algebraic Riccati equation:

$$\mathbf{A}^\top \mathbf{P} + \mathbf{P} \mathbf{A} - \mathbf{P} \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^\top \mathbf{P} + \mathbf{Q} = 0,$$

with $\mathbf{P}$ being the unique positive semi-definite solution.

6.4.1. Controller dimensioning

Since our model is non-linear the first step is to linearize it around an equilibrium point following the procedure outlined in [Sec. 4.3]. The equilibrium values we chose for $x_{1,\text{eq}}$ and $x_{3,\text{eq}}$ are 0.2 and 0, respectively. We selected

0.2 instead of zero due to a controllability issue: since the thrust force is modeled as proportional to the squared motor speed, linearizing around zero would result in the input coefficient being proportional to $2 \cdot w_{eq}$, which, for $x_1 = 0$, would unequivocally be zero.

The numerical matrices obtained with this procedure are:

$$A = \begin{bmatrix} 0 & 1.0000 & 0 & 0 & 0 & 0 \\ -1.0639 & -0.0760 & 0 & 0 & 0.0036 & 0.0032 \\ 0 & 0 & 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 0 & -30.7706 & -0.0029 & 0.0032 \\ 0 & 0 & 0 & 0 & -5.4875 & 0 \\ 0 & 0 & 0 & 0 & 0 & -5.4071 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 72.9329 & 0 \\ 0 & 72.9329 \end{bmatrix}$$

The second step is to enlarge the system with the integrators to ensure zero steady state error. The enlarging is allowed since the number of input is equal to the number of outputs and there are no invariant zeros in the origin (i.e. no derivative action)

The enlarged matrices are:

$$A_{\text{tilde}} = \begin{bmatrix} 0 & 1.0000 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1.0639 & -0.0760 & 0 & 0 & 0.0036 & 0.0032 & 0 & 0 \\ 0 & 0 & 0 & 1.0000 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -30.7706 & -0.0029 & 0.0032 & 0 & 0 \\ 0 & 0 & 0 & 0 & -5.4875 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -5.4071 & 0 & 0 \\ -1.0000 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1.0000 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B_{\text{tilde}} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 72.9329 & 0 \\ 0 & 72.9329 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

We now define the weight matrices for this enlarged system.

Q weights

The matrix Q weights the state variables. Higher values of its elements force the corresponding states to converge more quickly. For simplicity, we define Q as a diagonal matrix, so that the interpretation of each weight is straightforward: increasing a specific weight directly increases the convergence rate of the associated state.

Specifically:

- Weighting x_1 and x_3 accelerates the stabilization of the pitch and yaw angles.
- Weighting x_7 and x_8 promotes faster convergence of the accumulated errors on pitch and yaw.
- Weighting x_5 and x_6 encourages faster dynamics of the propellers. This means that the motor speed converge faster to equilibrium value. To have more aggressive control it is convenient to decrease the weight on the speed of the motor to have more room for control

R weights

The matrix R weights the control effort in the LQ cost function. As with Q , we initially adopted a diagonal R matrix for simplicity. Lower values in R allow for larger control efforts, while higher values penalize aggressive actions.

The system has two degrees of freedom: pitch and yaw. Input 1 directly controls pitch dynamics, while input 2 directly controls yaw. However, each input also indirectly affects the other axis through coupling torques: input 1 influences yaw, and input 2 influences pitch. By introducing off-diagonal components r_{12} and r_{21} in R , we explicitly weight these cross-effects. Increasing r_{12} and r_{21} reduces the controller's tendency to exploit coupling torques, effectively limiting control actions that rely on these indirect interactions.

Due to the large differences in the magnitudes of the states and inputs, it is good practice to normalize the weight matrices. To achieve this, we divide each weight by the square of the maximum expected value of the corresponding state or input. Specifically, we compute the normalized diagonal entries as:

$$Q_{\text{normalized},ii} = \frac{Q_{ii}}{(\max x_i)^2}, \quad R_{\text{normalized},jj} = \frac{R_{jj}}{(\max u_j)^2},$$

where $\max x_i$ and $\max u_j$ represent the maximum expected values of state x_i and input u_j , respectively.

Once the gain matrices are defined, to ensure the asymptotic stability of the LQ controller we check two key conditions:

1. The system is fully controllable:

$$\text{rank}(\mathcal{C}(A_{\text{tilde}}, B_{\text{tilde}})) = n,$$

where n is the dimension of A , and $\mathcal{C}(A, B)$ is the controllability matrix.

2. The system is fully observable (with respect to Q):

$$\text{rank}(\mathcal{O}(A_{\text{tilde}}, C_q)) = n, \quad \text{where} \quad C_q^\top C_q = Q,$$

and $\mathcal{O}(A, C_q)$ is the observability matrix.

The conditions are verified, thus solving the Riccati equation we obtain the gain K .

6.4.2. Results

As expected, the closed-loop scheme performs very well in converging to the linearization equilibrium and in tracking small reference steps as shown in [Fig. 35]. Notably, the controller also remains effective for larger-magnitude steps. This can be interpreted as evidence that the system's nonlinearities have limited impact within the operating range, and thanks to the robustness properties of LQ control, the controller is able to operate successfully across the full range of pitch angles $[-\frac{\pi}{4}, \frac{\pi}{4}]$ and yaw angles $[0, 2\pi]$.

We further investigated how the system's behavior changes when varying the weights in the Q and R matrices. The resulting system responses are shown in [Fig. 35]. We observe that, even when the response is fast and precise, small oscillations appear. This can be explained by the controller acting too aggressively, particularly in exploiting the coupling torques between pitch and yaw. As one input changes, the controller immediately adjusts the other input to compensate, leading to a loop of small compensatory oscillations.

This phenomenon is further exploited by increasing the control action as shown in [Fig. 36]. From the simulation, i.e. in ideal environment with no noise, increasing the control effort improve the response and does not introduce any oscillation, while on the real system increasing the control effort introduces bigger and bigger oscillations as the control action follows the noise too much.

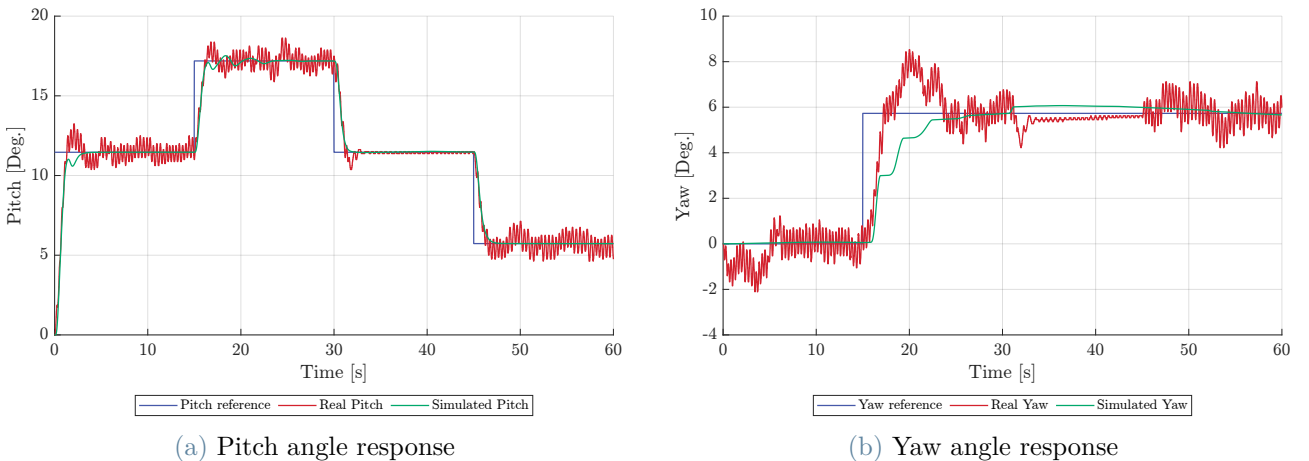


Figure 35: LQ controller response when tracking a trajectory near the linearization point with less aggressive control.

Weight matrices used: $Q = \text{diag}(100, 5, 100, 1, 0.1, 0.1, 100, 100)$, $R = \text{diag}(0.1, 0.1)$.

Finally we tried to follow a time varying trajectory far from the linearization point to test the capabilities of the controller even if this controller is not thought theoretically to move too far from the linearization point. As

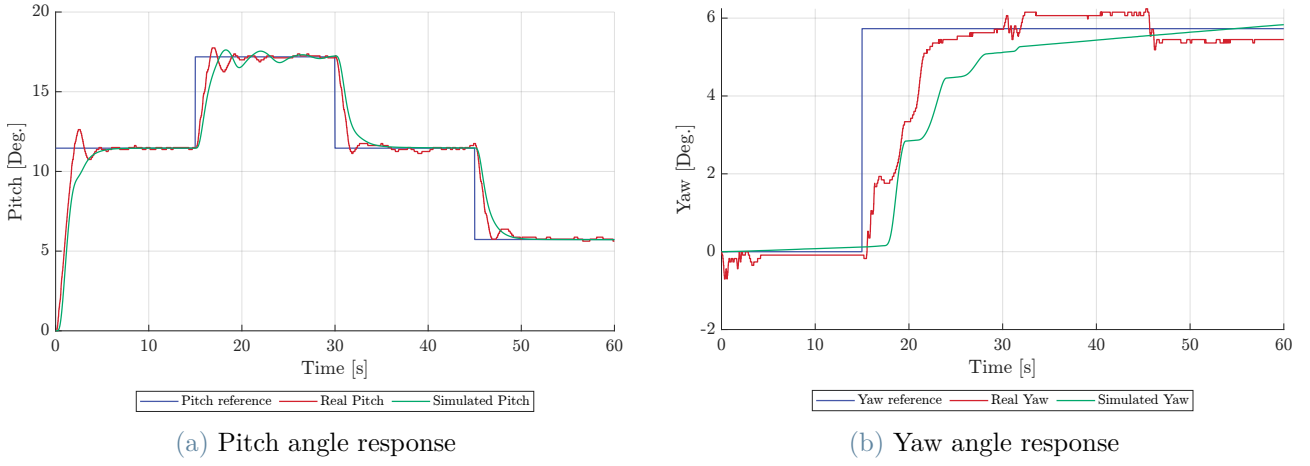


Figure 36: LQ controller response when tracking a trajectory close to the linearization point with aggressive control.

Weight matrices used: $Q = \text{diag}(100, 5, 100, 10, 0.1, 0.1, 500, 500)$, $R = \text{diag}(0.05, 0.05)$.

shown in [Fig. 37] these test gave satisfactory results both in simulation and on the real system. in the next section we will dimension a controller capable of performing this task from theory and we'll show the great results it is able to achieve

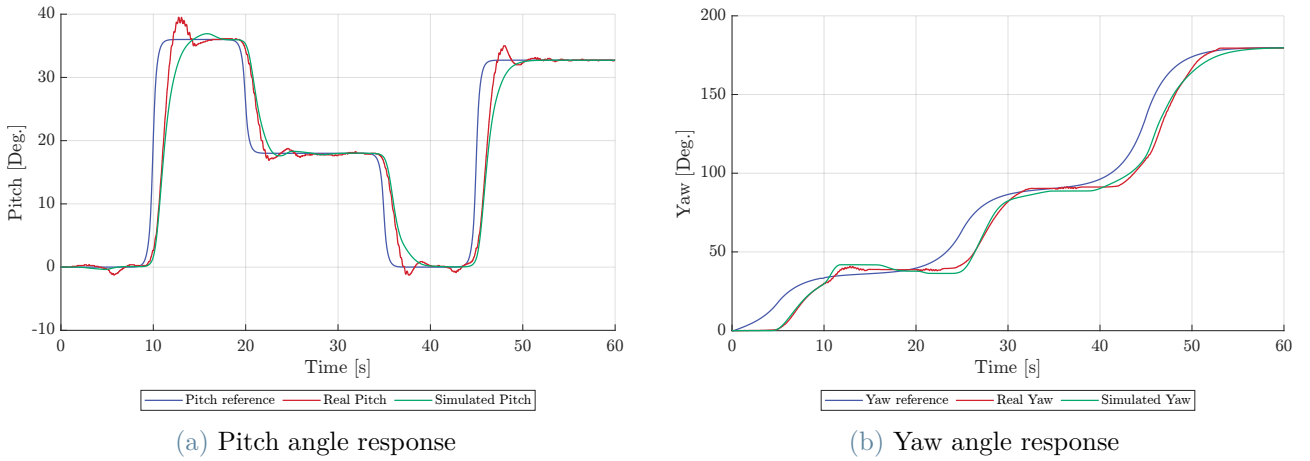


Figure 37: LQ controller performance for tracking a time-dependent trajectory.

Weight matrices used: $Q = \text{diag}(100, 5, 100, 1, 1, 1, 100, 100)$, $R = \text{diag}(0.1, 0.1)$.

6.5. LQ-TV Control

As we have seen, the LQ controller is not able to track time-dependent trajectories on our system, primarily due to the system's nonlinear nature. A solution to this limitation is to implement LQ control for a time-varying system; that is, we consider the system to be linear with time-varying state-space matrices.

To perform this task, we need to know the reference trajectory in advance and compute the optimal input required to follow it. The procedure to obtain the optimal input is detailed in Sec. 6.5.1.

6.5.1. Optimal Trajectory

We designed two trajectories to track:

- A piecewise trajectory, consisting of steps of varying magnitude switching at a given frequency Fig. ??
- A sinusoidal trajectory, with its frequency periodically increasing over time (see Fig. Y).

The desired rates of change (i.e., the speeds) of the pitch and yaw angles can be obtained by computing the derivatives of the respective trajectories. Naturally, for the piecewise trajectory, direct differentiation is not possible because the reference undergoes instantaneous jumps, which would imply infinite speed. To address this, we approximate the piecewise trajectory using cubic polynomials to smooth the transitions.

Given the known trajectories, we formulate an optimization problem to compute the optimal input:

$$\min_{\mathbf{u}(t)} \int_0^T [(\mathbf{x}(t) - \mathbf{x}_{\text{eq}}(t))^\top Q(\mathbf{x}(t) - \mathbf{x}_{\text{eq}}(t)) + \mathbf{u}(t)^\top R \mathbf{u}(t)] dt,$$

where the decision variable is the input $\mathbf{u}(t)$, and $\mathbf{x}_{\text{eq}}(t)$ is the desired time-varying trajectory. This optimization is subject to:

- The system dynamics.
- Physical constraints on both the states and the inputs.

Since we neither know nor have an easy way to estimate the motor speeds required to achieve $\mathbf{x}_{\text{eq}}(t)$, we leave the motor speeds as optimization variables. To reflect this in the cost function, we set the entries q_5 and q_6 of the Q matrix to zero, effectively removing penalties on those states.

A key drawback in the computation of the optimal input sequence arises from the limitations of the optimization problem formulation. Specifically, it is not possible to incorporate the exact nonlinear model, particularly the friction effects, into the optimization.

The friction force in the system always opposes the direction of motion and is modeled using a constant coefficient μ multiplied by the sign of the yaw speed x_4 , i.e., $\mu \text{sign}(x_4)$. However, including the discontinuous sign function within the optimization problem results in a non-smooth, non-convex formulation, making it too complex for the numerical optimizer, which would fail to find a feasible solution.

To overcome this issue, we approximate the sign function using a smooth hyperbolic tangent:

$$\text{sign}(x_4) \approx \tanh(100 x_4),$$

where the factor 100 ensures a steep transition near zero, closely mimicking the behavior of the sign function. Although this approximation is accurate near most operating points, it introduces slight deviations from the true model. As a result, the computed optimal input sequence is not perfectly aligned with the physical system, introducing small disturbances or mismatches in the control action.

6.5.2. Controller dimensioning

Once the optimal trajectory $\mathbf{x}_{\text{eq}}(t)$ is known, the control design process becomes fully analogous to the LQ procedure described in Sec. 6.4.1.

The key difference is that the matrix P in the Riccati equation is no longer constant (as in the time-invariant case) but becomes a time-dependent matrix obtained by solving a *Differential Riccati Equation* (DRE). Specifically, at each time instant, the system matrices $A(t)$ and $B(t)$ are computed by linearizing the nonlinear dynamics around the instantaneous equilibrium point:

$$A(t) = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}_{\text{eq}}(t), \mathbf{u}_{\text{eq}}(t)}, \quad B(t) = \left. \frac{\partial f}{\partial \mathbf{u}} \right|_{\mathbf{x}_{\text{eq}}(t), \mathbf{u}_{\text{eq}}(t)}.$$

The DRE is given by:

$$-\dot{P}(t) = A(t)^\top P(t) + P(t)A(t) - P(t)B(t)R(t)^{-1}B(t)^\top P(t) + Q(t),$$

with the final condition $P(T) = P_T$. Solving this equation backward in time yields the time-dependent matrix $P(t)$, which is then used to compute the time-varying feedback gain:

$$K(t) = R(t)^{-1}B(t)^\top P(t).$$

6.5.3. Results

This control law achieves exceptional results both in simulation and in real-world testing, although small disturbances are introduced due to the approximation used when computing the optimal input sequence.

Using diagonal Q and R matrices causes the system to suffer slightly from the coupling between yaw and pitch. As shown in figure 38,39 and 40, we compare the trajectory tracking performance when allowing more control effort on pitch versus on yaw.

From the pitch plot, we clearly observe that when equal control authority is given to both motors, the pitch responds very sensitively to the coupling torque and tends to correct it quickly, introducing small oscillations. In contrast, when more control power is given to the yaw motor, the pitch motor cannot immediately counteract the coupling torque, causing the trajectory to slightly overshoot and take more time to converge precisely. The best trajectory tracking is obtained when more control authority is allocated to the pitch motor and less to the yaw.

Looking at the yaw plot, we observe that the trajectory is less sensitive to changes in control effort compared to pitch. This is mainly due to the higher dissipative terms present in the yaw dynamics, particularly the damping. On simulation the control action is extremely balanced and even changing the weights in Q and R introduces non perceivable differences

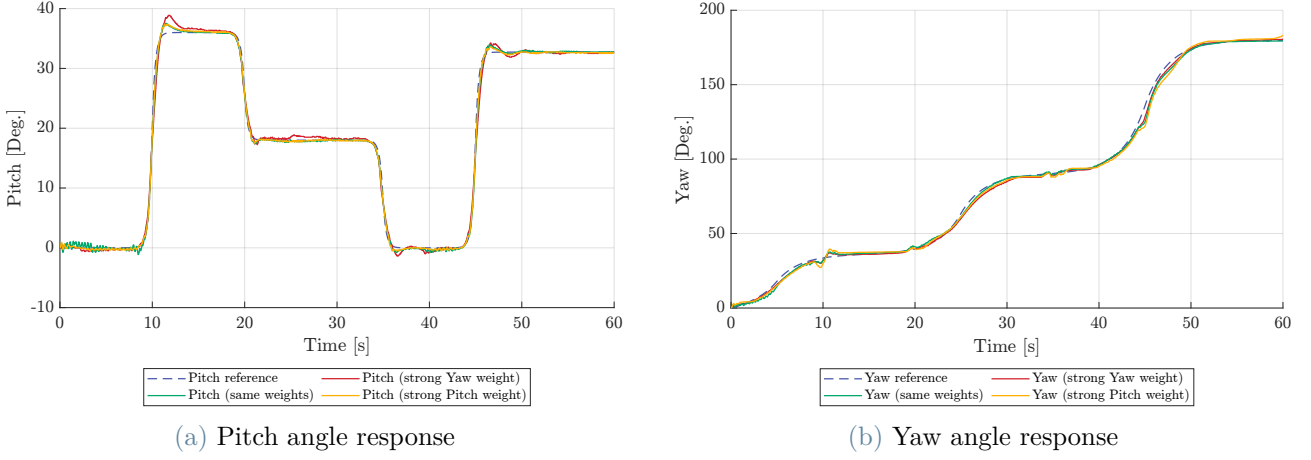


Figure 38: Real system response using LQ-TV control to follow a piecewise trajectory. Comparison of pitch and yaw behavior under different control efforts.

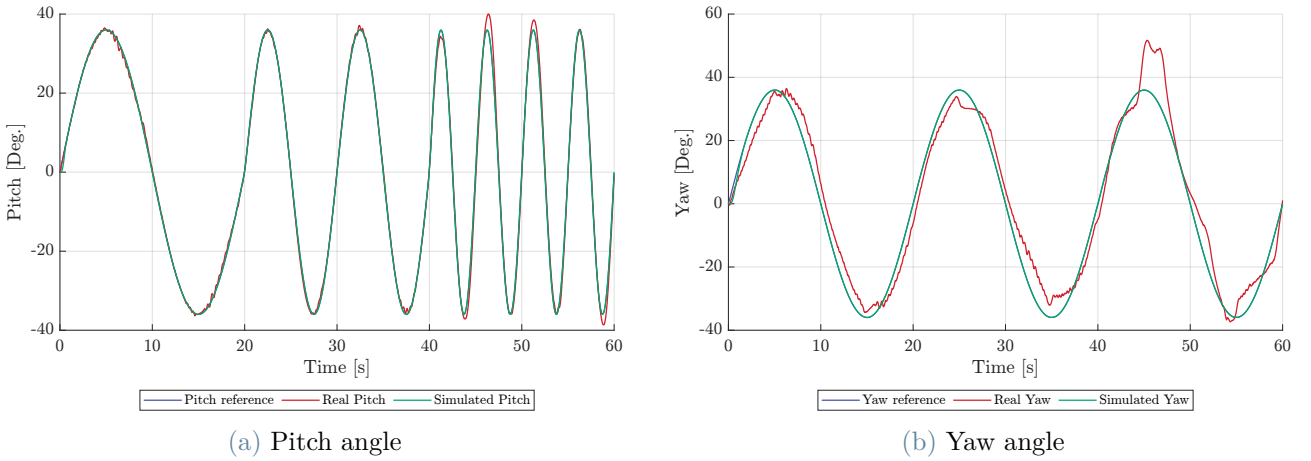


Figure 39: Real vs simulation LQ-TV control to follow a sine trajectory.

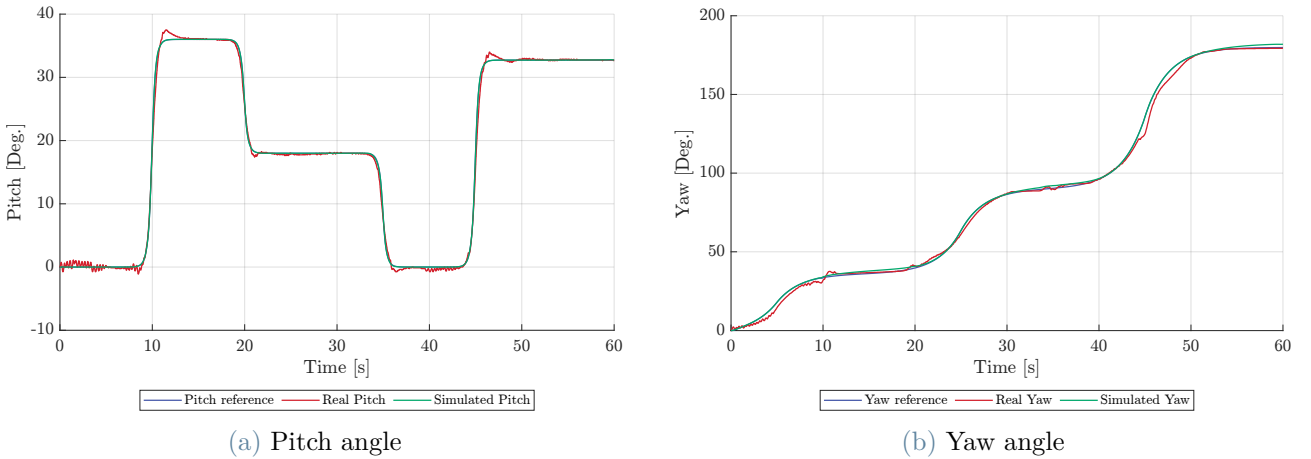


Figure 40: Real vs simulation LQ-TV control to follow a piecewise trajectory.

6.6. LQG control

The main idea behind Linear Quadratic Gaussian (LQG) control is to combine the optimal control capabilities of the Linear Quadratic (LQ) controller with the optimal state estimation provided by the Kalman Filter (KF). Although the mathematical model of our system is highly accurate and the measured signals are reliable, it remains a valuable and insightful exercise to explore the system's performance under the assumption that not all state variables are directly measurable and must instead be estimated.

6.6.1. Kalman Filter

The Kalman Filter provides an optimal, recursive method for estimating the internal state of a linear system subject to process and measurement noise. We assume the system evolves according to:

$$\mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k + \mathbf{w}_k, \quad \mathbf{w}_k \sim \mathcal{N}(0, Q_{\text{KF}}),$$

and the measurements follow:

$$\mathbf{y}_k = C\mathbf{x}_k + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(0, R_{\text{KF}}),$$

where:

- Q_{KF} is the process (state) noise covariance matrix,
- R_{KF} is the measurement noise covariance matrix.

The recursive estimated state update can be written compactly as:

$$\hat{\mathbf{x}}_{k+1} = A\hat{\mathbf{x}}_k + B\mathbf{u}_k + L(\mathbf{y}_k - C\hat{\mathbf{x}}_k),$$

where:

- $\hat{\mathbf{x}}_k$ is the estimated state at time k ,
- L is the Kalman gain matrix,
- $\mathbf{y}_k - C\hat{\mathbf{x}}_k$ is the innovation (the difference between measured and predicted output).

To compute the optimal gain L , one can solve an equivalent Linear Quadratic (LQ) problem on the dual system, as guaranteed by the separation principle. Specifically, the Kalman gain can be computed as:

$$L = [\text{lqr}(A^\top, C^\top, Q_{\text{KF}}, R_{\text{KF}})]^\top,$$

where $\text{lqr}(\cdot)$ denotes the standard LQ solution function.

Convergence Conditions

For the Kalman Filter to converge asymptotically, two key conditions must be satisfied:

1. The pair (A, B_q) must be observable, where $B_q^\top B_q = Q_{\text{KF}}$.
2. The pair (A, C) must be controllable.

These dual conditions ensure that:

- The state noise (propagated via B_q) can be reconstructed from the system outputs,
- The estimation error dynamics can be stabilized via the Kalman gain L .

6.6.2. Kalman Filter dimensioning

To construct the Kalman Filter, we define the process and measurement noise covariance matrices, Q_{KF} and R_{KF} , respectively.

We assume that the process noise and measurement noise are uncorrelated. Under this assumption, both Q_{KF} and R_{KF} can be modeled as diagonal matrices:

$$Q_{\text{KF}} = \text{diag}(q_1, q_2, q_3, q_4, q_5, q_6), \quad R_{\text{KF}} = \text{diag}(r_1, r_2, r_3, r_4, r_5, r_6).$$

The individual diagonal entries can be interpreted as follows:

- **Process noise covariance Q_{KF} :** higher values reflect lower confidence in the mathematical model of the system.
 - q_1 (associated with $\dot{x}_1$): set low ($\sim 10^{-1}$), since x_1 is obtained by direct integration of x_2 .
 - q_2 (associated with $\dot{x}_2$): slightly higher ($\sim 5 \cdot 10^{-1}$), reflecting moderate trust in the pitch dynamics model.
 - q_3 (associated with $\dot{x}_3$): set low ($\sim 10^{-1}$), for reasons analogous to q_1 .
 - q_4 (associated with $\dot{x}_4$): set high ($\sim 5 \cdot 10^1$), due to model uncertainty, particularly related to friction in yaw dynamics.

- q_5, q_6 (associated with propeller dynamics): set low ($\sim 10^{-1}$), as the mechanical model of the motors is highly accurate.
- **Measurement noise covariance R_{KF} :** higher values indicate lower confidence in the measurements. These values are chosen based on the observed magnitude of sensor noise.
 - r_1, r_3 (pitch and yaw angles): $\sim 10^{-1}$.
 - r_2, r_4 (pitch and yaw angular velocities): $\sim 10^0$.
 - r_5, r_6 (motor speeds): $\sim 5 \cdot 10^0$.

We implemented three configurations of the Kalman filter:

1. **Case 1: Measuring all states.**

The simplest configuration is to measure all relevant state variables and verify how accurately the Kalman Filter can track them. This serves as a reference case, where minimal estimation effort is required due to full observability of the system. The results are shown in Fig. 44.

2. **Case 2: Measuring both pitch and yaw angles.**

In this case, only the pitch and yaw angles are measured, and the remaining states must be estimated. The measurement noise covariance matrix becomes:

$$R_{KF} = \text{diag}(r_1, r_3), \quad r_1 = r_3 = 10^{-1}.$$

Estimating R_{KF} is straightforward, as its entries reflect the observed sensor noise levels.

In contrast, tuning Q_{KF} requires more care, as it encodes the level of trust in the system model. A trial-and-error approach was adopted, adjusting the entries based on model reliability and state observability. The final values used in the simulation are:

$$Q_{KF} = \text{diag}(\dots), \quad R_{KF} = \text{diag}(10^{-1}, 10^{-1}).$$

The results for the real system under this configuration are shown in Fig. 43.

3. **Case 3: Measuring only the yaw angle.**

This configuration limits measurements to the yaw angle only. The measurement matrix simplifies to:

$$R_{KF} = \text{diag}(r_3), \quad r_3 = 10^{-1}.$$

Since fewer states are directly observed, the filter must rely more heavily on the system model. Accordingly, Q_{KF} must reflect reduced confidence in unmeasured components. Estimating the pitch angle becomes especially challenging: from the model, we know that x_1 (pitch angle) influences yaw dynamics only through a sinusoidal function, making its estimation less accurate.

This limitation is clearly illustrated in Fig. 42, which compares the true and estimated pitch signals.

The final weights used in this configuration are:

$$Q_{KF} = \text{diag}(\dots), \quad R_{KF} = \text{diag}(10^{-1}).$$

Finally, seen the obvious difficulties for the system to follow the trajectory using only the yaw measurement, we realized a simpler reference to track to better test the results. [Fig. 41]

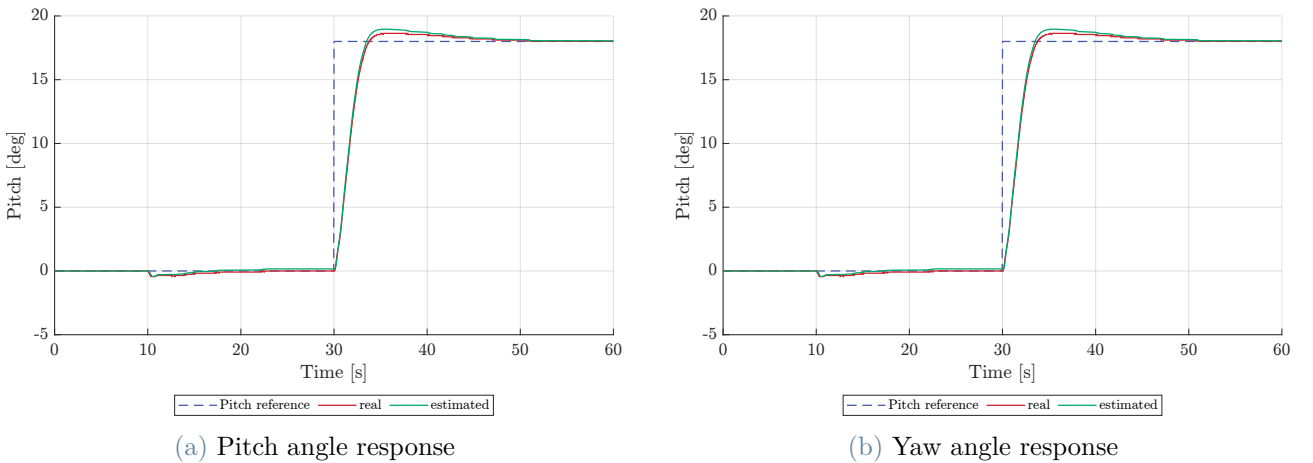
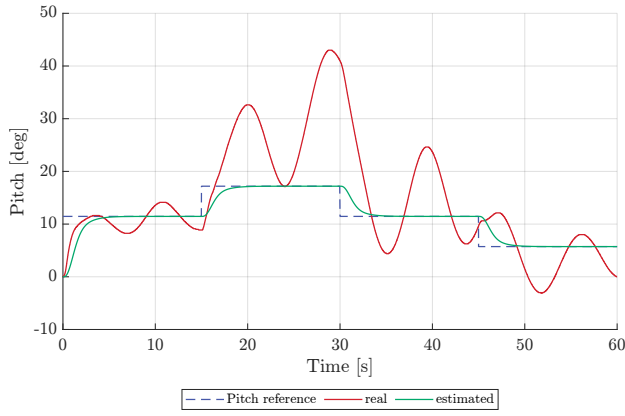
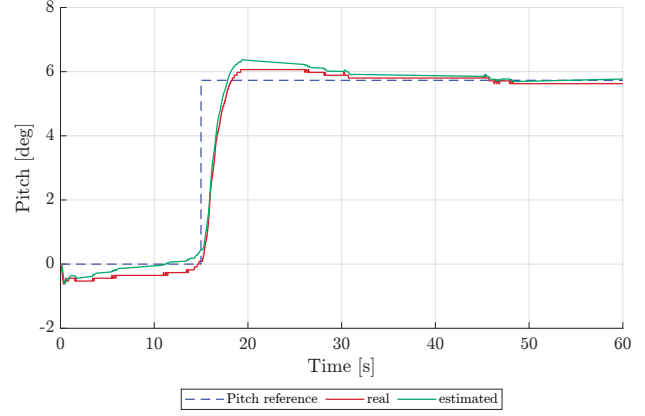


Figure 41: LQG controller performance for tracking a constant reference around the linearization equilibrium. Measuring only yaw angle

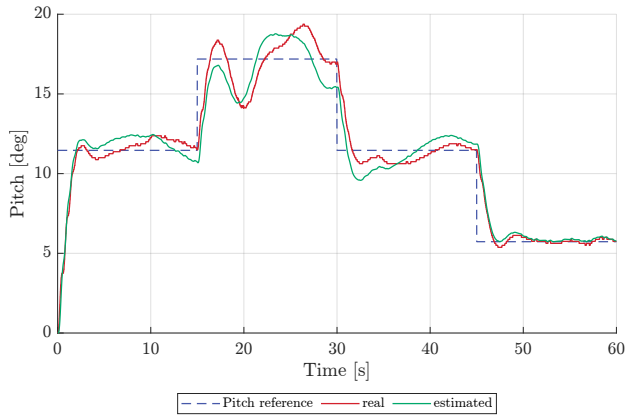


(a) Pitch angle response

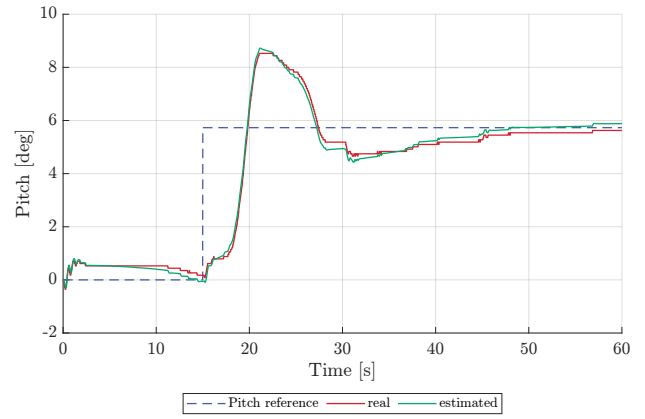


(b) Yaw angle response

Figure 42: LQG controller performance for tracking a reference around the linearization equilibrium. Measuring only yaw angle

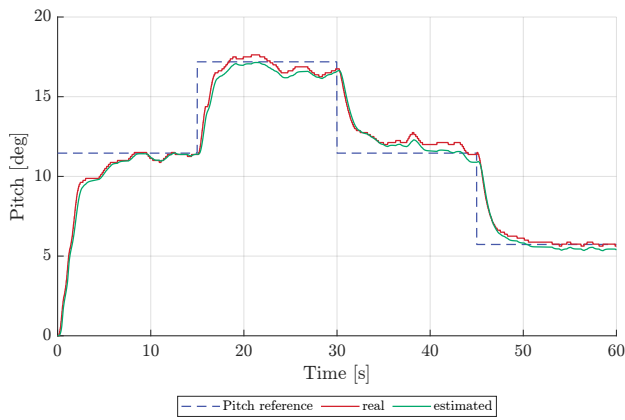


(a) Pitch angle response

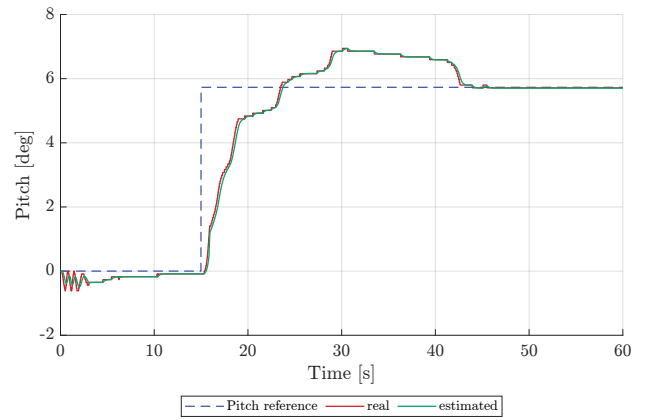


(b) Yaw angle response

Figure 43: LQG controller performance for tracking a reference around the linearization equilibrium. Measuring only yaw and pitch angles



(a) Pitch angle response



(b) Yaw angle response

Figure 44: LQG controller performance for tracking a reference around the linearization equilibrium. Measuring all the states

6.7. Quasi-Infinite Horizon Nonlinear MPC with Stability Guarantees

In this section, we detail the implementation of a nonlinear Model Predictive Control (MPC) scheme, specifically the quasi-infinite horizon variant with guaranteed stability, following the methodology described in [2]. This approach ensures nominal stability for a nonlinear system regular enough, the system is defined by the state equation:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}).$$

At each discrete time step, the applied control input is the first element of the solution to the following open-loop optimal control problem, formulated at time t with initial state $\mathbf{x}(t)$:

$$\min_{\mathbf{u}(\cdot)} J(\mathbf{x}(t), \mathbf{u}(\cdot)), \quad (29)$$

where the cost functional J is defined as:

$$J(\mathbf{u}(\cdot), t) = \int_t^{t+T_p} \left(\|\mathbf{x}(\tau)\|_Q^2 + \|\mathbf{u}(\tau)\|_R^2 \right) d\tau + \|\mathbf{x}(t+T_p)\|_P^2, \quad (30)$$

subject to:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)), \quad (31a)$$

$$\mathbf{u}(t) \in \mathcal{U}, \quad (31b)$$

$$\mathbf{x}(t+T_p) \in \mathcal{X}_\alpha, \quad (31c)$$

where Q and R are positive-definite, symmetric weighting matrices; T_p is the finite prediction horizon.

The terminal constraint and terminal region are designed offline and represent key components in ensuring nominal asymptotic stability. The paper outlines the following design procedure:

1. **Design a stabilizing controller** K such that the linearized system under this controller is asymptotically stable around the equilibrium point.
2. **Select a decay rate** κ such that $\kappa < -\max \Re(\lambda(A_K))$, where $A_K = A + BK$. Then solve the Lyapunov equation:

$$(A_K + \kappa I)^T P + P(A_K + \kappa I) = -(Q + \kappa^2 K^T R K)$$

to compute the matrix P .

3. **Determine the largest scalar** α_1 such that the control law $u = Kx$ satisfies the input constraint $u \in \mathcal{U}$ for all $x \in \mathcal{X}_{\alpha_1}$, where:

$$\mathcal{X}_{\alpha_1} = \{x \in \mathbb{R}^n : x^T P x \leq \alpha_1\}$$

4. **Find the largest scalar** $\alpha \in (0, \alpha_1]$ such that the following inequality holds:

$$L_\phi \leq \frac{\kappa \lambda_{\min}(P)}{\|P\|}$$

Here, L_ϕ represents the maximum deviation between the nonlinear model and its linearization.

6.7.1. Simulation and Real-Time Implementation

In this case, we adopt an approach in which the propellers are regulated through inner control loops. Consequently, the MPC design is based on the model presented in Equation ??, excluding the friction term, as the inclusion of the sign function would introduce a discontinuity.

The controller for the linearized system has been designed using the Linear Quadratic (LQ) method. This design yields the following controller and lead to the following parameters:

$$K = \begin{bmatrix} -2.9286 & -5.4734 & 3.0421 & 3.2222 \\ -2.7612 & -4.9633 & -3.2780 & -3.4641 \end{bmatrix}$$

$$P = \begin{bmatrix} 17.9861 & -4.5384 & -0.5338 & -0.3406 \\ -4.5384 & 6.6051 & 0.1209 & -0.0657 \\ -0.5338 & 0.1209 & 152.2465 & -139.9239 \\ -0.3406 & -0.0657 & -139.9239 & 139.7152 \end{bmatrix}$$

$$\alpha = 1.2298$$

The MPC controller was implemented in simulation using **CasADi** to solve the underlying optimization problem at each timestep. The simulation results were promising. As shown in the figure 45a, starting from an initial offset, the MPC controller is able to drive the system back to equilibrium rapidly. The chosen parameters include a prediction horizon of 7 steps and a sampling time of 0.1 seconds.

It is worth noting that the yaw angle does not fully converge to zero. This discrepancy is due to the absence of a friction term in the MPC model. One possible solution to address this limitation is to design the MPC for an extended system that includes integrators on both the yaw and pitch states.

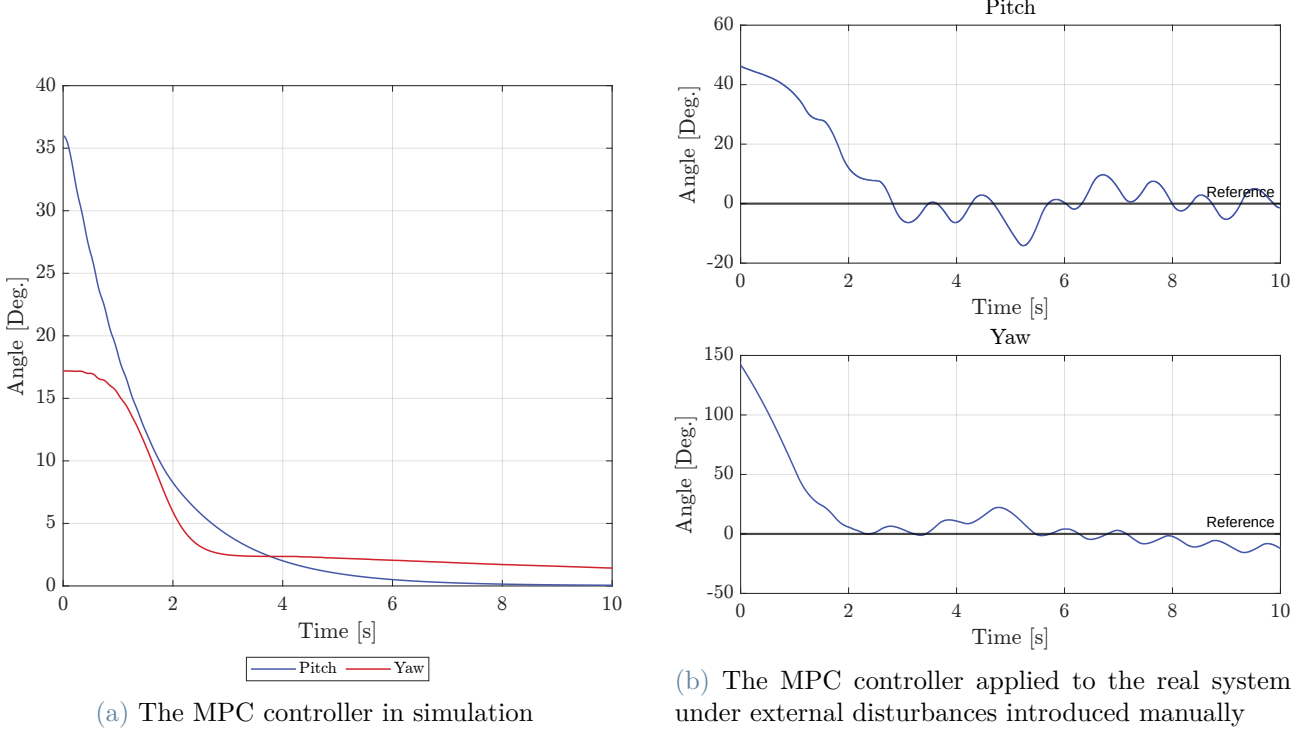


Figure 45: MPC controller in simulation and in reality

In the real-world implementation, several modifications were necessary. For instance, it was not feasible to use **CasADi** as a solver on the platform. Due to time constraints, only a limited real-time test of the MPC controller was conducted, and the results were not satisfactory. The plots in figure 45b indicate that, although the system can be driven near the equilibrium point at the origin, it exhibits persistent oscillations around it. This behavior is consistent with what was observed in other control strategies when the control input was excessively aggressive, this means that a better tuning parameters was needed, e.g. Q and R matrices.

7. Controller performance comparisons

7.1. 1 DOF performance comparison

From the data gathered, a few metrics were computed to assess the quality of the control action and to compare the controller architectures in a quantitative matter. For tracking performance, RMSE (Root Mean Squared Error), was selected. It is defined in the following manner:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N |y_{ref}(i) - y_{meas}(i)|^2}$$

Where N is the total number of samples, y_{ref} is the reference pitch trajectory, and y_{meas} the measured pitch from the real simulation.

The settling time[1] was calculated using an $\epsilon = 1\%$. Given that the fuzzy Error PID undershoots, a single metric was conceived that measures the percentage overshoot or undershoot in respect to the steps in the reference pitch trajectory. The RMS Effort measures the Root Mean Square of the control effort modulated by

each controller. It is computed in the following fashion:

$$RMS_{effort} = \sqrt{\frac{1}{N} \sum_{i=1}^N (v_{m0}(i)^2 + v_{m1}(i)^2)}$$

Where N is the total number of samples, v_{m0} is the voltage of motor 0, and v_{m1} the motor voltage of motor 1.

Architecture	RMSE Step	RMSE Sine	Settling Time (s)	Average Peak Over/Under	RMS Effort Step	RMS Effort Sine
Analytic PID	10.757	16.823	–	–	6.636	8.338
PD	4.887	13.903	3.804	0.250%	8.586	13.562
Fuzzy Error PID	4.399	13.835	2.704	5.248%	9.138	13.480
Fuzzy Position PID	4.481	16.052	5.054	8.335%	9.114	15.598

Table 3: Performance metrics of 1-DoF controllers in VTOL configuration. (lower is better)

From this table it can be assessed that for the 1 D.o.f. case the best performing controller is the Fuzzy Error PID as it has better reference tracking performance then all other controllers and a significantly lower settling time. It can be noticed that it requires on average a higher, but still comparable, control effort and also has a higher peak undershoot then the PD controller. The analytic PID, given it's very poor tracking performance (as can be seen in fig. 9), did not allow for the estimation of peak overshoot or undershoot or the settling time.

7.2. 2 DOF performance comparison

Given that the 2 DOF experiments were carried away in the Helicopter setup, the RMS definition is reworked:

$$RMS_{effort}^{\theta} = \sqrt{\frac{1}{N} \sum_{i=1}^N v_{m0}(i)^2} \text{ and } RMS_{effort}^{\psi} = \sqrt{\frac{1}{N} \sum_{i=1}^N v_{m1}(i)^2}$$

The settling time is defined as in the previous section but with a $\epsilon = 5\%$ because since it's 2 DOF the steady state behaviour is subject to more noise then the 1 DOF case. Also the definition of settling time here is therefore different then the definition used for the design parameter of the LMI that bears the same name. Considering trajectory tracking it's clear LQTV is the best approach, we therefore only compare LQ and LMI for stabilisation around their linearisation point.

Architecture	RMSE Pitch	RMSE Yaw	Settling Time Pitch (s)	Settling Time Yaw (s)	RMS Effort Pitch	RMS Effort Yaw
LMI	2.211	1.268	7.990	7.384	3.495	6.509
LQ	1.662	1.251	1.794	8.620	2.687	6.510

Table 4: Performance metrics of 2-DoF controllers in Helicopter configuration (lower is better).

As we can see it's clear that LQ is a better control strategy in this case than LMI, leading to a faster response with less control effort.

References

- [1] P. Bolzern, R. Scattolini, and N. Schiavoni. *Fondamenti di controlli automatici*. Collana di istruzione scientifica. McGraw-Hill Companies, 2008.
- [2] H. Chen and F. Allgöwer. A quasi-infinite horizon nonlinear model predictive control scheme with guaranteed stability. *Automatica*, 34(10):1205–1217, 1998.
- [3] Ratiba Fellag and Mahmoud Belhocine. Comparative analysis of pid, fuzzy pid, and anfis controllers for 2-dof helicopter trajectory tracking: simulation and hardware implementation. *Archive of Mechanical Engineering*, 09 2024.
- [4] Quanser Inc. *AERO 2 User Manual*, 2022.